



SAPIENZA
UNIVERSITÀ DI ROMA

Approximate Nearest-Neighbor Search on the Tenstorrent Wormhole Architecture

Faculty of Information Engineering, Computer Science and Statistics
Master's Degree in Computer Science

Barbalata Radu Ionut

ID number 2002688

Advisor

Prof. Daniele De Sensi

Co-Advisor

Lorenzo Piarulli

Academic Year 2025/2026

Approximate Nearest-Neighbor Search on the Tenstorrent Wormhole Architecture
Master's Thesis. Sapienza University of Rome

© 2026 Barbalata Radu Ionut. All rights reserved

This thesis has been typeset by L^AT_EX and the Sapthesis class.

Author's email: Barbalata.2002688@studenti.uniroma1.it

*To all the iterations that didn't work,
for teaching me what would.*

Abstract

This thesis investigates approximate nearest-neighbor (ANN) search as a new application domain for the Tenstorrent architecture and presents, to the best of our knowledge, its first ANN implementation. The main ANN index families are first analyzed to identify the one that best matches Tenstorrent’s tile-based spatial architecture. Based on this analysis, an IVF-Flat search pipeline is implemented on a Wormhole n150, combining device-side coarse search, host-side scheduling of the lists selected by each 32-query batch, and distributed fine search with hierarchical top- k selection. On the glove-100-angular dataset, it is compared at matched Recall@10 with Faiss on 56- and 112-core CPU configurations and on an NVIDIA A100 system. The A100 achieves the highest throughput throughout, from 31.9 times that of Tenstorrent at 60% recall to 7.8 times at 95%. Tenstorrent is slower than both CPU configurations at low recall, but its throughput declines more slowly as search depth grows: at 95% recall it is 1.74 times faster than the 56-core CPU and on par with the 112-core CPU against the evaluated Faiss grid. For a complete 100,000-query workload including initialization, the A100 system uses the least energy at every recall target. The Tenstorrent system uses 2.3–3.8 times as much as the A100 system but 53–79% less than the CPU configurations, and its energy is dominated by its host, with the Wormhole device accounting for only 11–18% of it. Fine search accounts for 90–98% of the Tenstorrent search time at 512 lists, identifying the per-list data-movement and compute loop as the main target for future optimization.

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Problem Statement	2
1.3	Research Questions	3
1.4	Contributions	3
1.5	Thesis Organization	4
2	Background	5
2.1	Approximate Nearest-Neighbor Search	5
2.2	Inverted File Indexes	6
2.3	Graph-Based Indexes	9
2.3.1	HNSW: Hierarchical Navigable Small World	9
2.3.2	NGT: Neighborhood Graph and Tree	12
2.4	GPU-Specialized ANN Indexes	12
2.4.1	GPU IVF and Faiss	14
2.4.2	GPU Graph Search and CAGRA	14
2.5	Tenstorrent Architecture and Programming Model	16
2.6	Related Work	18
2.6.1	ANN Search on Specialized Accelerators	18
2.6.2	Applications of Tenstorrent Hardware	18
2.7	Suitability of ANN Indexes for Tenstorrent	19
2.7.1	IVF-Flat	19
2.7.2	Graph-Based Indexes	20
2.7.3	CAGRA	21
2.7.4	IVF-PQ and Quantized Indexes	22
2.7.5	ScaNN	24
2.8	Motivation for Selecting IVF-Flat	25
3	Design and Implementation	27
3.1	System Architecture	27
3.1.1	End-to-End Search Pipeline	27
3.1.2	Host-Device Work Partitioning	28
3.1.3	Running Example	29
3.2	Device-Oriented IVF Representation	29
3.2.1	Index Construction	29
3.2.2	Tiled Data Layout and Memory Placement	30

3.3	Coarse Search	33
3.4	Fine-Search Scheduling	34
3.4.1	Batch-Level List Union	34
3.4.2	Greedy Work Distribution	35
3.4.3	Worker Description	36
3.5	Fine Search and Result Reduction	37
3.5.1	Worker Pipeline	37
3.5.2	Hierarchical Top- k	38
3.6	Multi-Batch Execution	41
3.7	Implementation Constraints	42
3.8	Chapter Summary	42
4	Experimental Methodology	44
4.1	Evaluation Goals	44
4.2	Hardware and Software Configuration	44
4.3	Dataset and Index Parameters	45
4.4	Recall, Throughput, and Matching	46
4.5	Benchmark Procedure	47
4.6	Tenstorrent Stage and Device Profiling	48
4.7	Power and Energy Measurement	49
4.8	Chapter Summary	51
5	Experimental Results	52
5.1	CPU, GPU, and Tenstorrent Comparison	52
5.2	Impact of N_{list} and N_{probe}	54
5.3	Tenstorrent Stage Decomposition	57
5.3.1	Device-Kernel Timeline	57
5.4	Workload Energy	61
5.4.1	Workload Throughput and Energy	65
5.5	Summary of Results and Limitations	65
6	Conclusions and Future Work	68
6.1	Answers to the Research Questions	68
6.2	Limitations	70
6.3	Future Work	71
	Acknowledgements	73
	Bibliography	74

Chapter 1

Introduction

1.1 Motivation

Modern artificial-intelligence systems increasingly rely on external retrieval components to access information that is not contained directly in a model’s parameters. Retrieval-Augmented Generation, or RAG [14], is a common example: before generating an answer, the system retrieves documents or document fragments that are relevant to the input query and provides them as additional context to the language model.

The retrieval stage is commonly implemented using a vector database. Documents are transformed into a dataset of high-dimensional embedding vectors

$$\mathcal{D} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n\}, \quad \mathbf{x}_i \in \mathbb{R}^d, \quad (1.1)$$

while an incoming query is encoded as a vector $\mathbf{q} \in \mathbb{R}^d$ in the same embedding space. The database then searches for the k vectors in $\mathcal{D}$ that are most similar to $\mathbf{q}$. These vectors correspond to the documents that are expected to contain the most relevant information.

At large scale, vector retrieval can become an important systems bottleneck. A production service may need to process large numbers of queries over datasets containing millions or billions of vectors. Exhaustively comparing every query with every stored vector provides exact results, but its cost grows linearly with the dataset size. Approximate nearest-neighbor indexes reduce this cost by examining only a subset of the indexed vectors, trading a limited amount of recall for higher throughput and lower latency.

Most existing vector-search systems are optimized for conventional CPUs and GPUs. CPUs are effective for irregular graph traversal and branch-heavy algorithms, while GPUs provide high throughput for regular batched computations. However, both architectures may consume substantial power when serving large retrieval workloads. This motivates the investigation of alternative hardware architectures that can provide competitive search quality and performance with lower energy consumption under a clearly defined measurement boundary.

Tenstorrent processors are spatial many-core accelerators designed primarily for machine-learning workloads. They consist of grids of programmable Tensix cores connected through Networks-on-Chip and rely on explicitly managed data movement

and distributed local memory [28]. These characteristics make Tenstorrent promising for regular and highly parallel operations, but it is not immediately clear whether they are also suitable for approximate nearest-neighbor search.

This thesis investigates whether vector retrieval can be efficiently implemented on Tenstorrent hardware. The broader motivation is to evaluate whether the same energy-efficient architecture used for machine-learning inference can also support the retrieval stage of AI systems such as RAG. A successful implementation could reduce the need to execute retrieval and model inference on separate classes of hardware and could provide a lower-energy platform for large-scale vector search.

1.2 Problem Statement

Given the dataset $\mathcal{D}$ and query vector $\mathbf{q}$ introduced above, the nearest-neighbor search problem consists of retrieving the k vectors in $\mathcal{D}$ that are closest to $\mathbf{q}$ according to a chosen distance or similarity measure.

An exact implementation evaluates the distance between the query and every vector in the dataset. For a dataset of n vectors of dimension d , its computational complexity is $O(nd)$ per query, excluding the final top- k selection. Although this computation is regular and highly parallel, its cost becomes significant as n grows.

Approximate nearest-neighbor search reduces the number of evaluated vectors by using an index that identifies a smaller candidate set

$$\mathcal{D}_{\text{candidate}}(\mathbf{q}) \subset \mathcal{D}. \quad (1.2)$$

The final results are selected only from this candidate set. Consequently, some of the true nearest neighbors may be excluded before their distance from the query is evaluated. The central trade-off is therefore between search quality and execution cost: searching more candidates generally improves recall but also increases latency, memory traffic, energy consumption, and hardware utilization.

A formal definition of the exact and approximate versions of the nearest-neighbor search problem, together with the notation and recall metric used throughout the thesis, is given in Section 2.1.

The problem addressed by this thesis is the design and evaluation of an approximate nearest-neighbor search system for Tenstorrent hardware. The implementation must account for the architectural properties of Tensix cores, including tile-based computation, explicitly managed DRAM transfers, limited per-core local memory, distributed execution, and synchronization over the Network-on-Chip.

The thesis focuses on the following challenges:

- identifying an ANN index whose computation and memory-access patterns are compatible with Tenstorrent hardware;
- mapping coarse search, fine search, candidate selection, and top- k processing onto Tensix cores;
- organizing queries and indexed vectors to exploit tile-oriented batch execution;
- reducing irregular memory accesses and unnecessary host-device communication;

- evaluating the resulting trade-off among Recall@10, throughput, and energy consumption;
- identifying the main bottlenecks that limit the performance of the current implementation.

1.3 Research Questions

This thesis is guided by the following research questions:

1. Which approximate nearest-neighbor index structure is most suitable for the current Tenstorrent architecture and programming model?
2. Can an IVF-based ANN search pipeline be implemented efficiently using tile-oriented computation, explicit data movement, and distributed Tensix cores?
3. What Recall@10 and throughput can the Tenstorrent implementation achieve compared with multicore CPU systems and an NVIDIA A100 GPU system?
4. How does recorded accelerator and host energy compare across Tenstorrent, CPU, and GPU ANN implementations at approximately matched recall, and what are the limits of the measurement boundary?
5. Which stages of the Tenstorrent search pipeline dominate execution time, and which architectural or software limitations are responsible for the observed bottlenecks?

1.4 Contributions

The main contributions of this thesis are:

1. An analysis of the suitability of major ANN index families for the Tenstorrent architecture, including IVF, graph-based methods, CAGRA, IVF-PQ, and ScaNN.
2. The design and implementation of TTAISS (Tenstorrent AI Similarity Search), an IVF-Flat search pipeline on Tenstorrent hardware, including coarse centroid search, cluster selection, distributed fine search, and result processing. To the best of our knowledge, this is the first approximate nearest-neighbor search implementation for the Tenstorrent architecture.
3. A Tenstorrent-specific execution strategy that organizes work around batches of queries and selected cluster sets rather than independent query-cluster tasks.
4. A quality-normalized experimental comparison against two multicore CPU configurations and an NVIDIA A100 GPU system.

5. An analysis of recorded component energy across several recall operating points, with explicit attention to measurement boundaries.
6. A stage-level decomposition of the Tenstorrent search path, showing the relative contribution of host preparation, host-to-device transfers, coarse search, fine search, and output processing.
7. An identification of the main performance limitations of the implementation and a discussion of possible future optimizations and alternative ANN designs.

1.5 Thesis Organization

The remainder of this thesis is organized as follows. Chapter 2 introduces approximate nearest-neighbor search, reviews the main index families and GPU-specialized approaches, describes the Tenstorrent architecture, discusses related work on ANN accelerators and on Tenstorrent applications, and motivates the selection of IVF-Flat. Chapter 3 presents the design and implementation, including data layout, coarse search, fine-search scheduling, and device-side execution. Chapter 4 describes the experimental setup and metrics, and Chapter 5 compares the CPU, GPU, and Tenstorrent systems in terms of recall, throughput, and energy, and analyzes the Tenstorrent search path. The final chapter answers the research questions and outlines future work.

Chapter 2

Background

This chapter provides the background required for the rest of the thesis. It introduces the main families of algorithms used for approximate nearest-neighbor search, reviews GPU-specialized methods, and describes the Tenstorrent architecture and programming model. It then reviews related work, analyzes the suitability of each ANN index family for execution on Tenstorrent hardware, motivating the selection of IVF-Flat for the implementation developed in this thesis.

2.1 Approximate Nearest-Neighbor Search

Approximate nearest-neighbor search, commonly abbreviated as ANN or ANNS, addresses the problem of finding the vectors in a dataset that are most similar to a given query vector without necessarily examining every stored vector [12, 15]. ANN search is widely used in information retrieval, recommendation systems, computer vision, semantic search, and retrieval-augmented generation.

Let

$$\mathcal{D} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n\}, \quad \mathbf{x}_i \in \mathbb{R}^d, \quad (2.1)$$

be a dataset containing n vectors of dimension d , and let $\mathbf{q} \in \mathbb{R}^d$ be a query vector. Given a distance or similarity function $s(\mathbf{q}, \mathbf{x})$, the exact k -nearest-neighbor problem consists of finding the set

$$\text{NN}_k(\mathbf{q}) = \arg \max_{\mathcal{S} \subseteq \mathcal{D}, |\mathcal{S}|=k} \sum_{\mathbf{x} \in \mathcal{S}} s(\mathbf{q}, \mathbf{x}). \quad (2.2)$$

Depending on the application, s may represent the inner product, cosine similarity, or the negative Euclidean distance. Using the negative Euclidean distance makes maximizing s equivalent to minimizing distance, so distance- and similarity-based formulations are used interchangeably throughout this thesis.

For normalized vectors, cosine similarity is equivalent to the inner product. If the dataset vectors are stored as the columns of

$$D = [\mathbf{x}_1 \quad \mathbf{x}_2 \quad \cdots \quad \mathbf{x}_n] \in \mathbb{R}^{d \times n}, \quad (2.3)$$

then all query–dataset similarities can be computed as

$$\mathbf{r} = \mathbf{q}^T D \in \mathbb{R}^{1 \times n}. \quad (2.4)$$

The k largest values in $\mathbf{r}$ are subsequently selected. The similarity-computation stage requires $O(nd)$ arithmetic operations. Maintaining the best k elements using a bounded heap requires $O(n \log k)$ time.

Exhaustive search is highly regular and can be implemented efficiently using matrix multiplication. Nevertheless, its cost increases linearly with the number of stored vectors, making approximate indexing attractive for large-scale datasets [12, 15]. Exhaustive search becomes expensive for datasets containing hundreds of millions or billions of high-dimensional vectors, particularly when many queries must be served under limited latency or energy budgets.

ANN indexes reduce the amount of work by searching only a query-dependent subset of the dataset. They therefore trade exactness for lower latency and higher throughput. Search quality is commonly measured through recall. For a query $\mathbf{q}$, Recall@ k can be defined as

$$\text{Recall@}k(\mathbf{q}) = \frac{|\text{ANN}_k(\mathbf{q}) \cap \text{NN}_k(\mathbf{q})|}{k}, \quad (2.5)$$

where $\text{ANN}_k(\mathbf{q})$ is the approximate result and $\text{NN}_k(\mathbf{q})$ is the exact ground truth.

The objective of an ANN system is not simply to minimize the number of visited vectors. Instead, it aims to achieve a favorable trade-off among recall, throughput, latency, memory consumption, index-construction cost, and energy usage. The percentage of the dataset examined depends on the workload and index configuration and is not fixed across ANN algorithms.

2.2 Inverted File Indexes

The inverted file index, usually referred to as IVF, is one of the most widely used index structures for large-scale vector search. It is implemented by systems such as Faiss and can be combined with compression techniques including product quantization [12]. The idea of partitioning a collection into inverted lists and searching only the most promising ones originates in text retrieval and was carried over to visual search [26]; its combination with product quantization for nearest-neighbor search was introduced by Jégou et al. [10].

IVF partitions the vector space into regions through a coarse quantizer. The coarse quantizer is commonly trained using k -means clustering, producing a set of centroid vectors

$$\mathcal{C} = \{\mathbf{c}_1, \mathbf{c}_2, \dots, \mathbf{c}_{N_{\text{list}}}\}. \quad (2.6)$$

Each dataset vector is assigned to its closest centroid:

$$a(\mathbf{x}) = \arg \min_{j \in \{1, \dots, N_{\text{list}}\}} \|\mathbf{x} - \mathbf{c}_j\|_2^2. \quad (2.7)$$

The vectors assigned to centroid $\mathbf{c}_j$ form an inverted list

$$L_j = \{\mathbf{x} \in \mathcal{D} \mid a(\mathbf{x}) = j\}. \quad (2.8)$$

The resulting partition can be interpreted as a Voronoi decomposition of the vector space. Every region contains the points for which a particular centroid is the

closest coarse representative. Figure 2.1 illustrates this partition: vectors located inside the same colored region are assigned to the same centroid, represented by the central point of that cell. For example, a vector located in the upper-left region is inserted into the inverted list associated with the upper-left centroid, even when it lies close to the boundary of an adjacent cell.

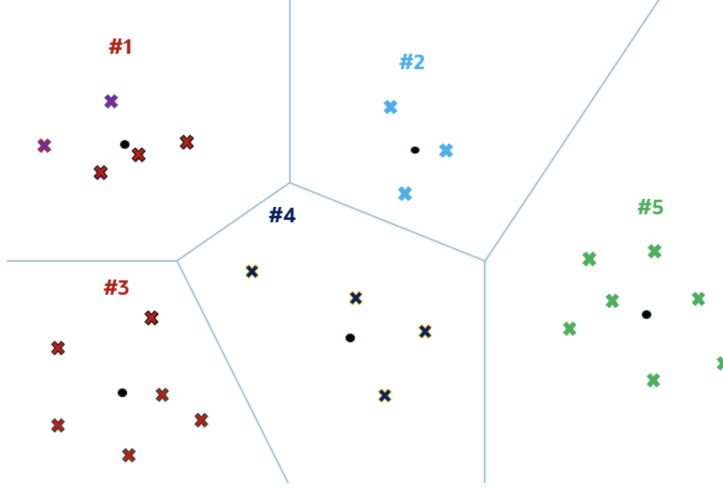


Figure 2.1. Example of a vector space partitioned into Voronoi cells. Each cell is associated with one coarse centroid, and each database vector is assigned to the inverted list of its closest centroid. Adapted from Oracle Database documentation [21].

Search is divided into a coarse quantization stage and a fine-search stage over the selected inverted lists [12]. During coarse search, the query is compared with the N_{list} centroids. The closest N_{probe} centroids are selected. Fine search then evaluates the vectors contained in the corresponding inverted lists:

$$\mathcal{D}_{\text{candidate}}(\mathbf{q}) = \bigcup_{j \in P(\mathbf{q})} L_j, \quad (2.9)$$

where $P(\mathbf{q})$ is the set of lists selected for query $\mathbf{q}$ and

$$|P(\mathbf{q})| = N_{\text{probe}}. \quad (2.10)$$

The main IVF parameters are:

- N_{list} , the number of coarse clusters or inverted lists into which the dataset is partitioned;
- N_{probe} , the number of inverted lists searched for each query.

Increasing N_{probe} generally increases recall because a larger portion of the dataset is examined, but it also increases execution time and memory traffic. Increasing N_{list} produces smaller clusters and can reduce the number of candidates per probed list, although it makes the coarse-search stage more expensive and may produce load imbalance when cluster sizes are not uniform.

The main source of approximation in IVF is the partition-boundary problem. A vector may be very close to the query while belonging to a Voronoi cell whose centroid was not selected during coarse search. In this case, the vector is excluded from the candidate set before its exact distance from the query is computed.

Figure 2.2 shows a concrete example of this behavior. The dataset is partitioned into five Voronoi cells, and the query $\mathbf{q}$ belongs to the red cell. The vectors labeled from #1 to #5 are the five exact nearest neighbors of the query. Four of these vectors belong to the selected red cell and are evaluated during fine search. However, vector #2 lies immediately across the boundary in the neighboring blue cell. Because that cell is not probed, vector #2 is not included in $\mathcal{D}_{\text{candidate}}(\mathbf{q})$, even though it belongs to the exact top-5 result.

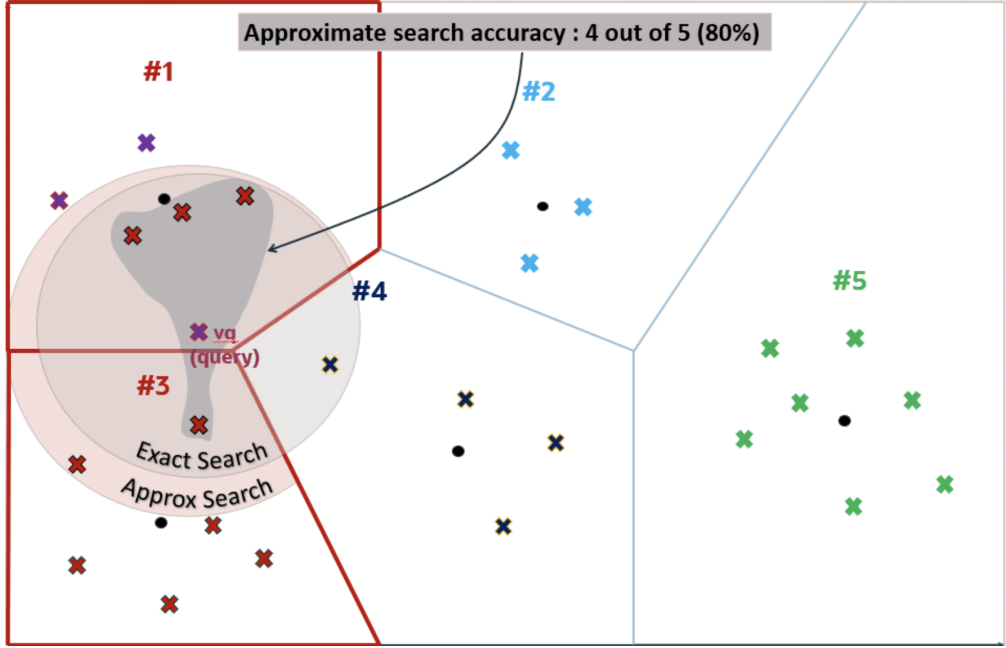


Figure 2.2. Example of a missed nearest neighbor in IVF search. The vectors labeled from #1 to #5 form the exact top-5 result. Vector #2 lies in an unprobed neighboring Voronoi cell and is therefore excluded from the approximate result, which retrieves four of the five exact nearest neighbors.

Using the recall definition introduced in Section 2.1, the approximate result in Figure 2.2 achieves

$$\text{Recall@5} = \frac{|\text{ANN}_5(\mathbf{q}) \cap \text{NN}_5(\mathbf{q})|}{5} = \frac{4}{5} = 0.8. \quad (2.11)$$

The missed vector is not caused by an inaccurate distance calculation. Its distance is never evaluated because coarse search excludes the inverted list containing it. Increasing N_{probe} may include the neighboring blue cell and recover vector #2, but it also increases the number of candidate vectors, distance computations, and memory transfers.

Despite this limitation, IVF exposes substantial parallelism. Coarse distances can be computed independently, selected clusters can be searched concurrently, and

distance computation within each cluster is regular. Vectors in an inverted list can also be stored contiguously, providing more predictable memory accesses than pointer-based index structures.

IVF-Flat stores the original vectors inside each inverted list. Consequently, the distances evaluated during fine search are exact with respect to the selected candidates; approximation is introduced only by the coarse partitioning and the selection of a subset of the lists.

Several extensions reduce the memory footprint or the amount of fine-search work. IVF-PQ stores product-quantized codes instead of complete vectors, while related methods compute approximate distances in the compressed domain [12]. ScaNN combines search-space pruning with quantization techniques, including anisotropic vector quantization for maximum inner-product search [8]. These approaches can significantly reduce memory traffic but introduce additional codebook lookups, approximate-distance computations, and often a re-ranking stage.

2.3 Graph-Based Indexes

Graph-based ANN methods represent dataset vectors as vertices of a proximity graph. Edges connect vectors that are close according to the selected metric. At query time, search starts from one or more entry vertices and repeatedly moves toward neighboring vertices that appear closer to the query.

Unlike IVF, graph search does not define a fixed set of spatial partitions to visit. The visited region is determined dynamically by the distances observed during traversal. This can provide high recall while evaluating a relatively small fraction of the dataset, including points located across coarse partition boundaries.

Graph-based methods are particularly effective on CPUs because CPUs provide efficient branch execution, cache hierarchies optimized for irregular access, and SIMD instructions for computing distances between a query and a small set of candidate vectors. Their main disadvantage is that traversal is data-dependent: different queries visit different vertices, perform different numbers of iterations, and access different memory locations.

The two representative graph-based approaches considered in this chapter are HNSW, described in Section 2.3.1, and NGT, described in Section 2.3.2.

2.3.1 HNSW: Hierarchical Navigable Small World

Hierarchical Navigable Small World, or HNSW, organizes proximity connections into multiple layers [15]. The bottom layer contains all indexed vectors, while progressively higher layers contain increasingly sparse subsets.

The level assigned to each vector is selected probabilistically, with the probability of appearing at higher levels decreasing exponentially. This creates a hierarchy conceptually similar to a skip list: upper layers contain long-range connections that allow the search to move rapidly across the dataset, while lower layers provide increasingly local refinement [23].

Search begins from an entry point in the highest layer. At each level, the algorithm greedily follows neighbors that improve the distance to the query. Once no closer neighbor is found, the search continues from the current vertex in the

next lower layer. At the bottom layer, a wider best-first search is performed using a dynamically maintained candidate queue [15].

A simplified representation of the search procedure is:

1. select an entry vertex in the highest graph layer.
2. greedily traverse the current layer toward the query.
3. use the resulting vertex as the entry point for the layer below.
4. at the bottom layer, maintain a bounded set of promising candidates.
5. terminate when the remaining candidates cannot improve the current result set.

Figure 2.3 illustrates this hierarchical organization. The upper layers contain only a sparse subset of the indexed vectors and provide long-range edges that allow the search to move rapidly toward the region containing the query. After reaching a local minimum in one layer, the search descends to the corresponding vertex in the next denser layer. The final and most detailed exploration is performed in layer 0, which contains all indexed vectors. This hierarchy allows HNSW to combine coarse navigation in the upper layers with fine-grained neighbor exploration in the bottom layer.

The maximum number of connections per vertex and the candidate-list size control the memory, recall, and latency trade-offs. HNSW is known for strong high-recall performance but may require substantial memory because both the original vectors and graph adjacency lists must be stored.

From an accelerator perspective, HNSW presents several challenges:

- neighbor addresses depend on the currently visited vertex;
- adjacency lists and vectors may be distributed across memory;
- the search frontier and result queue are updated dynamically;
- different queries follow different paths and terminate after different numbers of steps;
- visited-set management requires additional random accesses and synchronization.

These characteristics do not prevent accelerator implementations, but they make it harder to obtain the regular, high-throughput execution typically associated with dense tensor operations. Unlike the IVF partitioning described in Section 2.2, HNSW can cross coarse spatial boundaries by traversing graph edges. The consequences of its irregular traversal for Tenstorrent are discussed in Section 2.7.2.

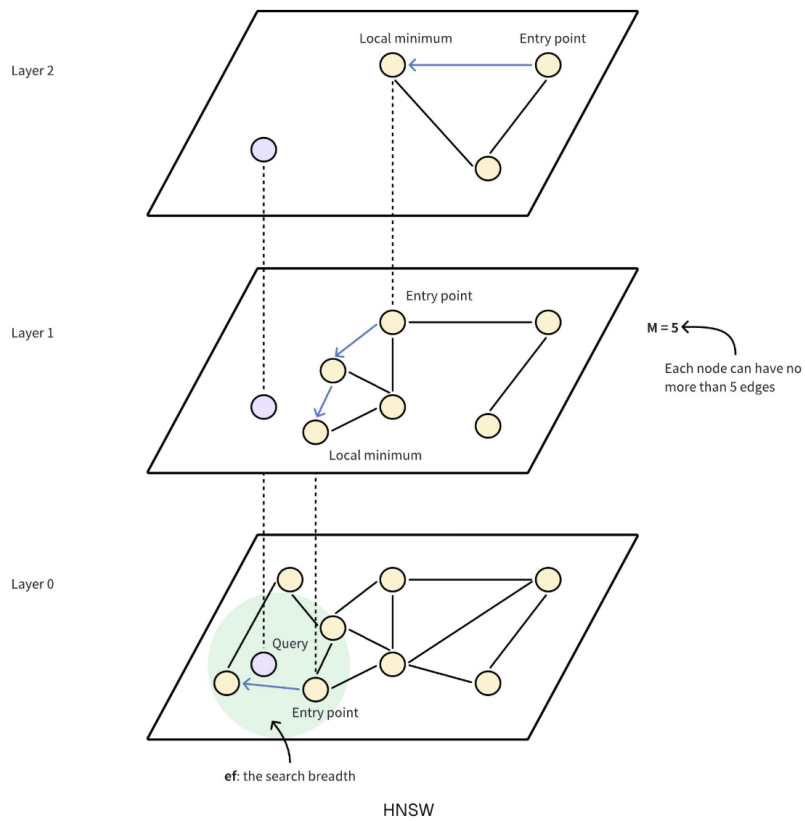


Figure 2.3. Hierarchical organization of an HNSW index. Sparse upper layers provide long-range navigation, while the dense bottom layer supports fine-grained nearest-neighbor search. Adapted from the Milvus HNSW documentation [16].

2.3.2 NGT: Neighborhood Graph and Tree

Neighborhood Graph and Tree, or NGT, combines a neighborhood graph with a separate tree-based structure used to obtain good entry points for graph traversal [9]. Conceptually, it is similar to HNSW in that both methods first try to move the search toward a promising region of the vector space before performing a more detailed local graph exploration.

The main difference lies in how this initial region is reached. In HNSW, the search uses the hierarchy of the graph itself: it starts from sparse upper layers and progressively descends toward the denser bottom layer. NGT instead uses a separate tree-based structure to locate one or more objects that are expected to be close to the query. These objects are then used as entry vertices for the neighborhood graph.

In this sense, the tree in NGT plays a role similar to the upper layers of HNSW, since both mechanisms reduce the need to begin graph traversal from an arbitrary point. However, the two approaches are structurally different. HNSW integrates coarse navigation and local search into a single hierarchical graph, whereas NGT separates the initial tree-based localization from the subsequent neighborhood-graph traversal.

Once the entry vertices have been identified, NGT explores the neighborhood graph by maintaining a candidate set ordered according to distance from the query. The most promising unexplored candidate is expanded, its neighboring vertices are evaluated, and sufficiently close neighbors are inserted into the candidate set. This process continues until the termination condition is satisfied, after which the best k objects found during traversal are returned.

Figure 2.4 summarizes the search process. The tree first performs a coarse localization of the query and selects the initial graph vertices. The neighborhood graph is then explored from these seeds to progressively refine the candidate set and obtain the approximate nearest neighbors.

As with HNSW, NGT can achieve high recall while evaluating only a subset of the dataset. Its execution is nevertheless irregular: the next vertices to visit depend on the query and on the distances observed during traversal. The search therefore involves non-contiguous graph accesses, dynamic candidate-set updates, and branch-dependent control flow. The additional tree traversal introduces another query-dependent stage before neighborhood exploration begins.

For this thesis, HNSW and NGT illustrate a common property of graph-based ANN methods: they reduce the number of distance computations by relying on irregular memory accesses and dynamic traversal decisions. These characteristics differ substantially from the regular, tile-oriented execution model targeted by the Tenstorrent architecture, and their suitability is discussed jointly in Section 2.7.2.

2.4 GPU-Specialized ANN Indexes

GPUs provide high arithmetic throughput and memory bandwidth, making them well-suited to regular batched distance computations. However, efficient ANN search on GPUs requires algorithms that expose sufficient parallelism and avoid frequent divergent branches or fine-grained dynamic memory operations.

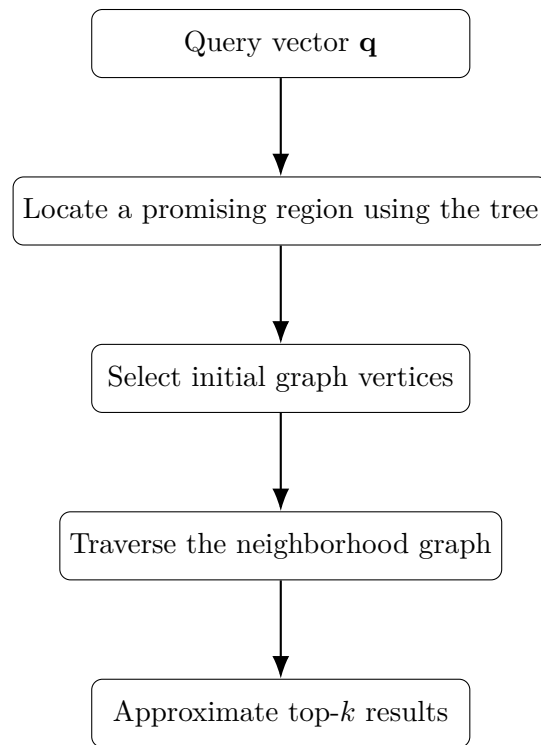


Figure 2.4. High-level NGT search process. The tree first identifies a promising region of the vector space and provides initial graph vertices. The neighborhood graph is then explored from these seeds to refine the search and obtain the approximate nearest neighbors. Original illustration based on [9].

2.4.1 GPU IVF and Faiss

IVF naturally maps to GPU execution because many queries, clusters, and candidate vectors can be processed concurrently. GPU implementations can parallelize both the coarse centroid comparisons and the fine-search distance calculations over selected clusters.

One important difficulty is selecting the smallest or largest k elements from a large set of candidate distances. A conventional heap performs data-dependent insertions and provides insufficient parallelism for efficient GPU execution.

The GPU implementation presented by Johnson et al. in Faiss introduces a register-resident selection method designed around GPU warps [12]. Candidate values are processed cooperatively by the threads of a warp, reducing global-memory traffic and avoiding the use of a conventional branch-heavy heap. The method is integrated with brute-force, IVF, and product-quantized search.

The resulting search pipeline uses the GPU effectively because:

- distance calculations are independent and data parallel;
- the same kernel can process large batches of queries;
- candidate vectors can be read through coalesced or block-oriented memory accesses;
- selection is performed using warp-level cooperation;
- intermediate results can remain in registers or shared memory.

The Faiss work demonstrates that the top- k stage, rather than only distance computation, must be designed specifically for the target architecture [12].

2.4.2 GPU Graph Search and CAGRA

Traditional proximity-graph indexes are difficult to execute efficiently on GPUs because each query follows a data-dependent path and maintains its own candidate and visited sets. CAGRA addresses this problem through a graph structure and traversal strategy designed specifically for highly parallel GPU execution [20].

CAGRA demonstrates that graph-based ANN search can be adapted to GPUs, but this requires substantially more than transferring a conventional CPU graph traversal to CUDA. Its design modifies both graph construction and query execution to expose bounded and predictable parallel work.

First, CAGRA constructs a single-layer proximity graph with a fixed out-degree, bounding the amount of work associated with each vertex expansion [20]. A bounded degree ensures that expanding one graph vertex always produces a known maximum number of neighbors, allowing their vector distances to be evaluated cooperatively by GPU threads. This is considerably more regular than a graph in which adjacency-list lengths vary arbitrarily.

Second, graph traversal is organized around fixed-capacity candidate and result structures rather than dynamically allocated CPU priority queues. Threads within a GPU block cooperate in expanding candidates, computing distances, filtering

previously visited vertices, and updating the candidate set. These structures must fit within the memory and synchronization model available to a GPU thread block.

Third, visited-node tracking is implemented using a bounded GPU-oriented structure. Unlike an exact dynamically growing CPU set, this representation accepts limited capacity and possible repeated visits in exchange for predictable memory usage and parallel lookup behavior. The search procedure is therefore designed to tolerate approximation not only in graph traversal but also in auxiliary bookkeeping.

Finally, the stopping policy and the number of expanded candidates are adapted to parallel execution. The algorithm must keep enough work available for the GPU while limiting divergent control flow between queries. These changes show that accelerator graph search requires the graph degree, candidate queue, visited set, work assignment, and termination condition to be co-designed with the target architecture [20].

CAGRA versus IVF on GPUs

Neither CAGRA nor IVF is universally superior on GPUs. The preferable index depends on the required recall, dataset size and distribution, query-batch size, available memory, and index-construction constraints.

IVF exposes regular and highly parallel execution because vectors within each selected inverted list can be read and evaluated in contiguous blocks. Its index structure is comparatively simple, and quantized variants such as IVF-PQ can reduce storage and device-memory bandwidth. However, high-recall search may require probing many inverted lists. As N_{probe} increases, the system evaluates more candidate vectors and approaches the cost of exhaustive search.

CAGRA instead uses graph connectivity to navigate toward promising areas of the dataset without scanning entire clusters. Its fixed-degree graph and cooperative GPU traversal are designed to provide high throughput at high recall [20]. This advantage comes with additional graph storage, more complex index construction, indirect vector accesses, and candidate and visited-state structures that must be managed during traversal.

CAGRA is therefore particularly attractive for high-recall, memory-resident GPU workloads when graph construction and storage costs are acceptable. IVF remains attractive when regular execution, simpler construction, predictable partitioning, support for datasets larger than available accelerator memory, or compatibility with compression are more important. The comparison is implementation-dependent: optimized quantized IVF variants may outperform graph-based methods at some high-recall operating points [24].

The approach used by CAGRA is highly relevant when considering graph search on Tenstorrent. It shows that accelerator execution is possible, but a direct port of a CPU graph algorithm is unlikely to be efficient. The graph structure, memory layout, candidate management, and traversal strategy must all be redesigned around the target accelerator.

2.5 Tenstorrent Architecture and Programming Model

Tenstorrent processors are spatial many-core accelerators whose Tensix cores form a two-dimensional grid connected through one or more Networks-on-Chip. Rather than relying primarily on hardware-managed caches and general-purpose threads, the programming model gives the developer explicit control over data placement, movement, core assignment, and synchronization.

Figure 2.5 shows the main components of a Tensix core. A core combines local L1 SRAM, RISC-V processors responsible for data movement and kernel control, and specialized compute engines. The data-movement processors issue transfers through the NoC, while the compute path includes a matrix engine for tile-oriented operations and a vector engine for element-wise and special-function operations. The local SRAM stores circular buffers, input and output tiles, intermediate values, runtime metadata, and other kernel state. Consequently, performance depends not only on arithmetic throughput but also on how effectively computation and data movement are pipelined through the limited local-memory capacity.

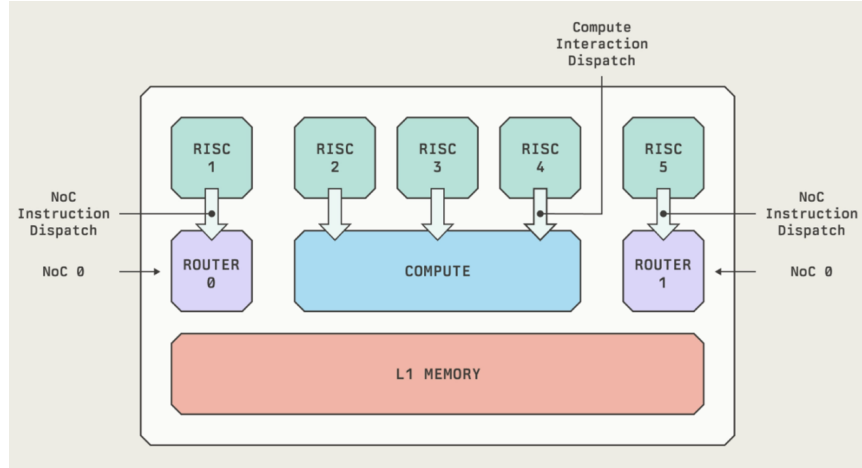


Figure 2.5. Simplified organization of a Tensix core, including its RISC-V control and data-movement processors, local L1 SRAM, NoC interfaces, and specialized compute engines. Adapted from the Tenstorrent architecture and TT-Metalium documentation [28].

Tenstorrent provides several programming layers with different levels of abstraction. At the highest level considered here, TT-NN provides a Python interface and a library of common tensor and machine-learning operations. It is suitable when an application can be expressed using operations already provided by the library, allowing the runtime to manage much of the device configuration and tensor placement.

TT-Lang provides an intermediate programming layer between TT-NN and TT-Metalium. It is a Python-based domain-specific language for implementing custom Tenstorrent operations while retaining explicit control over data movement, synchronization, and inter-core communication. It provides higher-level constructs such as tensor slices, blocks, and pipes, while remaining more flexible than the predefined operations available in TT-NN. TT-Lang was under active development

during this work, and the implementation presented in this thesis was developed directly using TT-Metalium.

TT-Metalium is the lower-level C++ programming interface used to define programs, allocate buffers, select Tensix cores, configure circular buffers, compile kernels, set runtime arguments, and explicitly schedule data movement. TT-Metalium exposes the architecture in sufficient detail to implement non-standard operations such as the ANN kernels developed in this thesis. A typical operation is built from reader, compute, and writer kernels and is first developed for one Tensix core before being distributed across multiple cores or devices.

TT-LLK, or the Low-Level Kernel layer, contains architecture-specific implementations and interfaces for the underlying Tensix compute engines. TT-Metalium compute APIs are commonly built on these lower-level kernels, while direct modification or extension of TT-LLK may be required when the available operations do not expose a needed hardware primitive. In this thesis, the limitations of the available top- k , comparison, and data-manipulation primitives influence which ANN algorithms can be implemented efficiently.

The software stack can therefore be viewed as

$$\text{TT-NN} \rightarrow \text{TT-Lang} \rightarrow \text{TT-Metalium} \rightarrow \text{TT-LLK} \rightarrow \text{Tensix hardware}, \quad (2.12)$$

where moving downward provides greater architectural control but also requires more explicit management of memory, synchronization, tilization, and kernel execution.

Most computation on a Tensix core is performed on tiles. The standard tile is a 32×32 block whose elements share the same data type. Tensors whose final dimensions are not multiples of the tile dimensions generally require padding or a different supported layout.

The relevant memory hierarchy includes:

- off-chip device DRAM, which provides the capacity required for large tensors and indexes;
- local L1 SRAM associated with each Tensix core, which provides lower-latency storage for tiles, intermediate values, and circular buffers;
- explicit transfers over the Network-on-Chip between DRAM interfaces and Tensix cores or among Tensix cores.

Data in DRAM is not accessed through a transparent cache. Device kernels explicitly request transfers using the NoC data-movement APIs. Therefore, efficient execution depends strongly on the regularity of the memory-access pattern, the size of each transfer, and the placement of data relative to the cores that consume it.

A typical TT-Metalium program uses three kinds of kernels:

- a reader kernel, responsible for transferring input data into local buffers;
- a compute kernel, which operates on tiles using the Tensix compute engine;
- a writer kernel, responsible for transferring results to their final location.

These kernels communicate through circular buffers in local SRAM. Circular buffers implement a producer-consumer pipeline: a reader reserves space and pushes input tiles, the compute kernel waits for available tiles and produces outputs, and the writer waits for output tiles before transferring them. Double buffering can overlap data movement with computation.

This execution model is particularly effective for workloads with:

- regular tile-based computation;
- statically partitionable work;
- predictable data movement;
- contiguous or block-oriented memory accesses;
- sufficient work per transferred tile;
- opportunities for batching and data reuse.

Conversely, workloads dominated by pointer chasing, fine-grained random access, dynamically sized data structures, and query-dependent control flow are more difficult to map efficiently. Such workloads may still be implementable, but often require transformations that increase the amount of work, memory traffic, or synchronization.

2.6 Related Work

2.6.1 ANN Search on Specialized Accelerators

Beyond CPUs and GPUs, ANN search has been mapped to FPGAs, TPUs, and dedicated hardware. FANNS co-designs the parameters of an IVF-PQ index and an FPGA accelerator for a given dataset and recall target, reporting speed-ups of up to 23.0 times over FPGA baselines and 37.2 times over CPU baselines [11]. TPU-KNN instead performs nearest-neighbor search on TPUs at close to peak FLOP/s: guided by a performance model that accounts for both memory and instruction bottlenecks, it replaces index traversal with dense matrix computation and outperforms GPU methods at similar recall [7]. ANNA proposes a specialized architecture for product-quantization-based ANN search [13]. TPU-KNN is the closest in spirit to this thesis: both target a matrix-oriented accelerator and trade additional arithmetic for regular, tile-friendly execution.

2.6.2 Applications of Tenstorrent Hardware

Published studies of Tenstorrent hardware beyond machine-learning inference have focused on dense numerical kernels. On the Grayskull e150, a Jacobi stencil reached performance similar to a Xeon Platinum CPU while using about five times less energy [4]. On the Wormhole, a two-dimensional FFT ran slower than a 24-core Xeon Platinum CPU but consumed about 2.8 times less energy [5], and a gravitational N -body simulation on a Wormhole n300 achieved more than a twofold speed-up and about twofold energy savings over an optimized CPU implementation [1].

These workloads have regular, data-independent access patterns. ANN search differs in that the data accessed by each query depends on a data-dependent list selection, list sizes are irregular, and results require a top- k selection rather than a dense output. To the best of our knowledge, no approximate nearest-neighbor search implementation for Tenstorrent hardware has been published, and this thesis presents the first.

2.7 Suitability of ANN Indexes for Tenstorrent

The suitability of an ANN index for Tenstorrent depends on more than its algorithmic complexity. An index that performs fewer distance computations may still execute inefficiently if it requires irregular memory accesses, dynamic priority queues, or frequent synchronization.

The principal criteria considered in this thesis are:

1. regularity of the computation;
2. spatial locality of the indexed vectors;
3. predictability and granularity of DRAM transfers;
4. ability to batch multiple queries;
5. amount of query-dependent control flow;
6. size and dynamism of intermediate data structures;
7. possibility of statically distributing work across Tensix cores.

2.7.1 IVF-Flat

IVF-Flat provides the most natural mapping among the considered ANN indexes. Both coarse and fine search are dominated by independent vector-distance calculations. The centroids form a dense matrix, while vectors assigned to the same cluster can be stored in contiguous regions of device DRAM.

Coarse search can be performed for a batch of queries by comparing a 32-query tile with tiles containing the centroid vectors. This arrangement matches the native tile dimensions and allows the queries to be reused while centroid blocks are streamed through the compute cores.

Fine search can then be partitioned by assigning selected clusters or groups of clusters to different workers. Each worker reads contiguous vector blocks, computes their distances from the corresponding queries, and produces a local set of candidates. Partial results are subsequently merged.

The implementation developed in this thesis does not treat each query-cluster pair as a completely independent GPU-style task. Instead, work is organized around batches of up to 32 queries and the union of the clusters selected for those queries. This allows the data layout and computation to remain compatible with tile-based execution.

The main difficulties are:

- inverted lists have different numbers of vectors;
- different queries may select overlapping or disjoint sets of lists;
- the amount of work increases with N_{probe} ;
- workers may receive unequal candidate-set sizes;
- local candidate sets must be merged into a global top- k ;
- cluster metadata and worker schedules must be generated and transferred efficiently.

These issues require explicit scheduling and load balancing, but the dominant computation remains regular enough to benefit from the Tensix architecture. As discussed in Section 2.8, this combination of properties made IVF-Flat the index selected for the implementation developed in this thesis.

This section provides only the architectural motivation for mapping IVF-Flat to Tenstorrent. The complete implementation, including the data layout, query batching, coarse-search kernels, fine-search scheduling, top- k processing, and host-device interaction, is described in Chapter 3.

2.7.2 Graph-Based Indexes

Graph-based indexes are less naturally aligned with the current Tenstorrent programming model. To expand one graph vertex, the implementation must first retrieve its adjacency list and then fetch the vectors associated with the listed neighbors. The addresses of these vectors are known only after the current vertex has been processed.

The resulting memory-access sequence is therefore indirect:

$$\text{vertex} \rightarrow \text{adjacency list} \rightarrow \text{neighbor identifiers} \rightarrow \text{neighbor vectors}. \quad (2.13)$$

The neighbor vectors are not guaranteed to be contiguous in DRAM. Transferring complete tiles may consequently fetch vectors that are not needed by the current traversal. Alternatively, issuing small row-major transfers can reduce the transferred volume but may underutilize DRAM and NoC bandwidth.

Graph traversal also requires:

- a dynamic candidate queue ordered by distance;
- a bounded result set;
- a representation of the previously visited vertices;
- repeated termination checks;
- synchronization when multiple workers cooperate on one query.

Batching does not completely solve the problem because different queries may visit unrelated regions and execute different numbers of iterations. Some queries may therefore finish while others continue traversing the graph, reducing the utilization of cores assigned to the batch.

Graph indexes should therefore not be described as impossible to implement on Tenstorrent. A possible implementation could use fixed-degree graphs, fixed-capacity candidate buffers, batched frontier expansion, and approximate visited sets. However, this would require a substantial redesign similar in spirit to CAGRA and was beyond the scope of this thesis.

2.7.3 CAGRA

CAGRA is designed for GPU execution, but its GPU-oriented design does not automatically make it efficient on Tenstorrent. Both architectures exploit parallelism, but their execution models and memory hierarchies differ.

The fixed graph degree in CAGRA is beneficial because it bounds the amount of work associated with one vertex expansion. Groups of 32 neighbors could, in principle, be arranged to match a Tensix tile dimension. Nevertheless, the identifiers still refer to vectors that may be located in unrelated DRAM regions. Efficiently gathering those vectors remains difficult.

The candidate queue and visited-state structure also require careful consideration. A GPU implementation can use registers, shared memory, and thread-block cooperation to maintain bounded structures. On Tenstorrent, each Tensix core has a limited amount of L1 SRAM that is also used for circular buffers, input tiles, output tiles, metadata, and program state. Reserving a large per-query candidate or visited structure would reduce the space available for double buffering and data reuse.

Placing these structures in device DRAM would increase capacity but introduce additional NoC transfers and latency on every graph iteration. Distributing them over multiple Tensix cores would require synchronization and ownership rules for concurrent updates.

A Tenstorrent-oriented graph index could still be explored in future work, for example by combining:

- a fixed graph degree compatible with tile dimensions;
- batched expansion of several query frontiers;
- a compact probabilistic visited-state representation;
- graph reordering to improve spatial locality;
- bounded iteration counts and fixed-capacity candidate queues.

Another possible Tenstorrent-specific design would group multiple dataset vectors into one logical graph vertex. For example, each logical vertex could contain a block of 32 vectors, allowing one block to align with a native tile dimension and enabling the distance calculations for those vectors to be performed together. The graph connecting these blocks could use a small fixed degree, such as four outgoing edges. Most edges could connect blocks representing nearby regions, while one edge could provide a longer-range connection intended to improve global navigability.

This organization would reduce the granularity mismatch between conventional graph traversal and tile-oriented execution. Fetching one graph vertex would provide enough vectors to occupy the compute engine, and a fixed low degree would bound the number of successor blocks considered during each expansion. It could also reduce adjacency metadata compared with representing every vector as an independent graph node.

However, grouping vectors changes the semantics of the graph. The 32 vectors contained in one block may not all be equally close to one another or to the query, and selecting a block may require evaluating many vectors that would not have been visited by a conventional graph search. Constructing balanced blocks with both good internal locality and useful inter-block edges would therefore become a separate indexing problem. The search would still require a candidate queue, a visited-block representation, query-dependent termination, and indirect DRAM accesses to successor blocks. Long-range edges would also need to be constructed carefully to improve navigability without causing excessive exploration. This design may better match Tensix execution than a conventional per-vector graph, but it does not eliminate the fundamental control-flow and memory-access challenges of graph-based ANN search.

Such an implementation would therefore constitute a separate research project rather than a direct extension of the IVF pipeline.

2.7.4 IVF-PQ and Quantized Indexes

Product quantization [10] divides a vector into m sub-vectors and replaces each sub-vector with the identifier of its closest codeword. A database vector is therefore represented using a short sequence of codebook indices rather than its original coordinates. Figure 2.6 illustrates this process: the original vector is divided into independent subspaces, each sub-vector is mapped to a codeword from its corresponding codebook, and the resulting codeword identifiers form the compressed representation.

For a query, lookup tables containing the distance between each query sub-vector and each codeword can be precomputed. As shown in Figure 2.6, the approximate distance to a compressed database vector is obtained by using its codeword identifiers to select one value from each lookup table and summing the selected values.

IVF-PQ preserves the coarse partitioning of IVF and therefore retains many of its favorable properties:

- encoded vectors can be stored contiguously within each inverted list;
- many codes can be evaluated independently;
- the reduced representation lowers DRAM capacity and bandwidth requirements;
- coarse search and cluster scheduling can reuse the IVF-Flat design.

The difficulty is not that the original vector must be fetched for every candidate. During the approximate stage, distances can be computed directly from the compressed codes. Original vectors are required only when an optional exact re-ranking

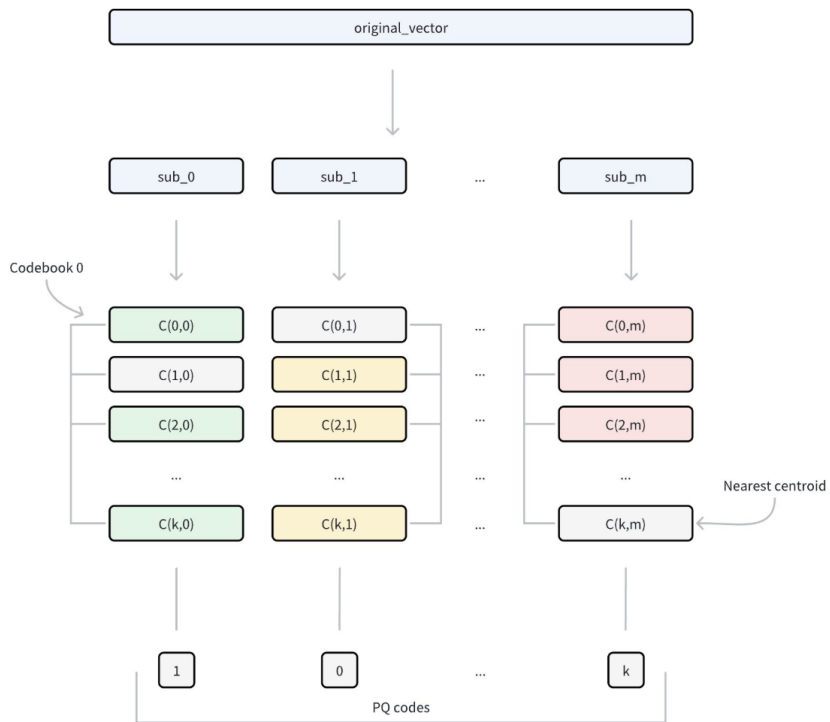


Figure 2.6. Overview of product quantization within an IVF-PQ index. Each vector is divided into sub-vectors and represented by codeword identifiers from independently trained subspace codebooks. Approximate distances are computed by accumulating values from query-specific lookup tables. Adapted from the Milvus IVF-PQ documentation [17].

stage is performed.

The main implementation challenges on Tenstorrent are:

- implementing efficient small-table lookups;
- decoding packed codeword identifiers;
- supporting integer and mixed-precision accumulation;
- arranging codebooks for efficient local reuse;
- gathering original vectors if exact re-ranking is enabled.

Although IVF-PQ reduces the amount of data stored in DRAM, its fine-search access pattern is less suitable for the current Tenstorrent implementation than the sequential reading of full vectors used by IVF-Flat. Each candidate code contains several packed codeword identifiers, and evaluating it requires decoding those identifiers and performing multiple indexed accesses into relatively small distance tables. These gathers are regular at the algorithmic level but do not directly correspond to the dense tile operations for which the available Tensix compute primitives are optimized.

The lookup tables could be copied into local SRAM and reused across many candidates, but each Tensix core would then need sufficient local capacity for the tables, circular buffers, candidate codes, intermediate distances, and top- k state. If the tables or decoded data cannot remain local, repeated accesses through device memory or the NoC may offset part of the bandwidth reduction obtained from compression. Packed-code extraction and mixed-precision accumulation would also require additional low-level kernels.

For these reasons, IVF-PQ was not considered a promising first index for the current implementation. This conclusion concerns its mapping to the present Tenstorrent software stack rather than the general efficiency of IVF-PQ. A specialized implementation with local lookup-table reuse, efficient packed-code decoding, and dedicated quantized-distance kernels could make IVF-PQ attractive in future work.

2.7.5 ScaNN

ScaNN combines partitioning, quantization, and re-ranking. Its anisotropic quantization objective gives greater importance to preserving components of the quantization residual that have a larger effect on the inner product [8].

A typical ScaNN-style search pipeline combines partition-based pruning, compressed candidate scoring, candidate selection, and optional exact re-ranking [8]:

1. partition-based pruning of the search space;
2. approximate scoring of a comparatively large candidate set using highly compressed representations;
3. selection of a sufficiently large set of promising candidates;
4. re-ranking of those candidates using their original vectors to obtain the final top- k results.

The partitioning and approximate-scoring stages offer parallelism and could potentially be mapped to Tensix cores. However, an efficient implementation would require optimized quantized-distance kernels and a sufficiently large intermediate candidate selection.

In the implementation considered in this thesis, the available top- k primitive from TT-LLK and the local-memory budget constrain the number of candidates that can be maintained efficiently inside one device program. Re-ranking also requires gathering original vectors whose identifiers were produced by the compressed search stage. Unless the dataset is reordered or the selected identifiers exhibit locality, these vectors may be distributed across device DRAM.

These issues do not make ScaNN fundamentally incompatible with Tenstorrent. They indicate that an efficient implementation would require additional support for large candidate selection, quantized lookups, and batched vector gathering. For this reason, ScaNN was considered beyond the scope of the first Tenstorrent ANN implementation.

2.8 Motivation for Selecting IVF-Flat

Section 2.7 compared IVF-Flat, graph-based indexes, CAGRA, IVF-PQ, and ScaNN against the same set of criteria: regularity of computation, spatial locality, DRAM-transfer granularity, batchability, query-dependent control flow, and the possibility of static work distribution. IVF-Flat was the only index for which all of these criteria were favorable without requiring a substantial architectural redesign, which is why it was selected for the implementation developed in this thesis.

Two further points reinforced this choice. First, IVF-Flat avoids compression-related approximation during candidate scoring: once a list has been selected, its vectors are compared with the query using their original representation. This simplifies correctness validation and allows the effects of N_{list} , N_{probe} , scheduling, and data movement to be studied directly without conflating them with quantization error.

Second, IVF still exposes challenges representative of real accelerator systems, including variable cluster sizes, load balancing, host-side scheduling, DRAM placement, NoC traffic, local top- k selection, and result merging, without requiring the fully dynamic execution model of graph-based search.

Table 2.1 summarizes the principal trade-offs of the index families considered in this chapter.

The selection of IVF-Flat should not be interpreted as evidence that other ANN indexes cannot execute on Tenstorrent. Instead, IVF-Flat is the index whose computation, memory layout, and scheduling requirements most directly match the capabilities of the current architecture and software stack. It provides a suitable starting point for studying the feasibility, performance, energy efficiency, and bottlenecks of ANN search on Tenstorrent hardware. Its complete implementation is presented in Chapter 3.

Table 2.1. Summary of the ANN indexes considered and their qualitative suitability for the current Tenstorrent programming model.

Index	Main advantage	Main limitation	Tenstorrent suitability
Flat	Regular exhaustive computation	Linear work in dataset size	Regular, but excessively expensive at scale
IVF-Flat	Contiguous candidate scans and tunable recall	May miss vectors in unprobed cells	High; selected for this thesis
HNSW	High recall with relatively few distance evaluations	Irregular traversal and substantial graph state	Low without substantial redesign
NGT	Tree-based seeding followed by graph refinement	Dynamic traversal and indirect memory accesses	Low without substantial redesign
CAGRA	GPU-oriented high-recall graph search	Complex traversal state and indirect accesses	Potentially feasible with a TT-specific redesign
IVF-PQ	Reduced storage and memory traffic	Packed decoding and lookup-table accesses	Moderate, but difficult with current primitives
ScaNN	Effective pruning, quantization, and re-ranking	Large intermediate selection and irregular re-ranking	Moderate to low with the current software stack

Chapter 3

Design and Implementation

3.1 System Architecture

This chapter presents the system design and implementation of a Tenstorrent-oriented IVF-Flat approximate nearest-neighbor search pipeline on a single-chip Tenstorrent Wormhole n150 device using TT-Metalium [28, 30]. IVF-Flat provides the underlying indexing algorithm; the contribution of this chapter is its mapping to the target accelerator. In particular, the design maps regular, fixed-shape operations, including centroid scoring and page-wise candidate scoring, onto the tile-oriented Tensix execution model, while explicitly handling the irregularity caused by query-dependent list selection and unequal inverted-list sizes; the choice of IVF-Flat itself is motivated in Section 2.7.

The implementation is organized around two principal device programs: a *coarse-search program*, which compares queries with the IVF centroids, and a *fine-search program*, which evaluates candidate vectors from the selected inverted lists and reduces the results to the final top- k . Host-side processing connects the two phases by selecting the N_{probe} lists returned by coarse search and constructing the work schedule used by the fine-search workers. Figure 3.1 gives an overview of the complete search; the tag on each step indicates the section of this chapter that describes it.

3.1.1 End-to-End Search Pipeline

A query is first compared with all N_{list} centroids, after which the N_{probe} most similar centroids identify the inverted lists to be searched. Candidate vectors contained in those lists are then compared with the query, and the final top- k scores and vector identifiers are selected from the resulting similarities.

The selected candidates are not copied into a single temporary matrix. Instead, database vectors remain grouped by inverted list in device memory and are streamed directly from the selected lists during fine search.

Queries are represented internally in groups containing between one and 32 valid queries, matching the row dimension of the 32×32 tile representation used by the Tensix compute kernels. A group containing fewer than 32 valid queries is completed with zero-valued padded rows. For a workload containing Q valid queries,

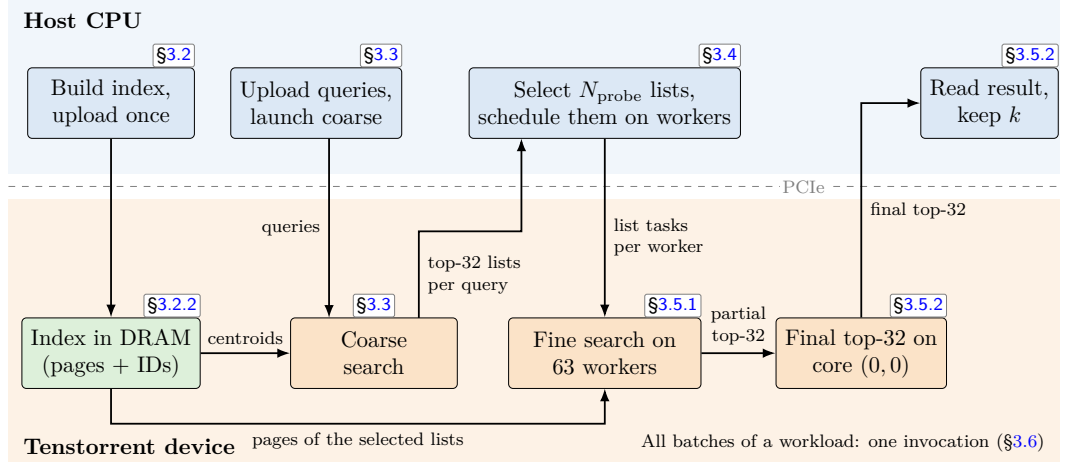


Figure 3.1. Map of the chapter. Host steps are on top and device steps below; the tag on each box gives the section that describes it. The index is uploaded once; every query workload then runs through the remaining steps.

the implementation uses

$$B = \left\lceil \frac{Q}{32} \right\rceil \quad (3.1)$$

internal query batches. The final batch is padded when Q is not divisible by 32. The number of valid rows is retained separately, so padded rows do not participate in coarse-result processing and do not contribute inverted lists to the fine-search schedule. They are also discarded when the final results are converted back to the host representation.

Although the implementation is described throughout the chapter in terms of one 32-query batch for clarity, the final system can process multiple such batches within one coarse-search and one fine-search invocation. The transition from the initial single-batch implementation to this multi-batch design is discussed in Section 3.6.

3.1.2 Host–Device Work Partitioning

The index data required by both search stages remain resident on the Tenstorrent device across query workloads, including the centroid matrix, database vectors grouped by inverted list, and the corresponding original vector identifiers. In contrast, query-dependent data, such as the query buffer, coarse-search outputs, selected-list metadata, worker schedules, partial results, and final outputs, are updated for each search workload.

A synchronization boundary separates the coarse and fine programs. After the device executes coarse search and returns a fixed-size set of centroid candidates for each valid query, the host performs the final N_{probe} selection, obtains the offset and padded length of each corresponding inverted list, groups the selected work at batch granularity, and constructs the per-worker fine-search descriptions. Only once this query-dependent schedule has been prepared is the fine-search program launched.

The selection of the N_{probe} lists and the steps that follow it run on the host rather than on the device because they require a set-like, dynamically sized data

structure. The lists selected by the up to 32 queries of a batch must be collected into a set without duplicates, whose size depends on how much the selections overlap, and the resulting lists must then be split into chunks, sorted by size, and distributed among the workers. Such structures fit the Tensix cores poorly: each core has only a limited amount of L1 memory, which the search already uses for its circular buffers, and the programming model is built around fixed-size tiles and streams rather than dynamically growing containers (Section 2.5). On the host, the same work is simple and cheap: it takes 7.8–10.7 ms per 10,000 queries at $N_{\text{list}} = 512$, at most 3% of the search time (Section 5.3).

The division of responsibilities between the host and the device corresponds to the two rows of Figure 3.1, and the synchronization boundary to the arrows that cross the PCIe line between them.

Each device program is decomposed into *reader*, *compute*, and *writer* kernels connected through circular buffers, allowing data movement and computation to overlap when dependencies and buffer capacity permit it. Throughout this chapter, reader, compute, and writer denote kernels executing on a Tensix core, whereas worker and aggregation core denote logical roles assigned to complete Tensix cores.

3.1.3 Running Example

To make the following sections easier to follow, all examples in this chapter use the same small configuration. It keeps the structure of the real search but replaces every size with a small one, so that each step can be drawn in full. Table 3.1 lists the example values next to the real ones. The eight example lists A–H hold 7, 3, 10, 4, 5, 2, 6, and 3 vectors, and the four queries of the batch select the lists q_0 : {A, C}, q_1 : {A, B}, q_2 : {C, E}, and q_3 : {A, D}; lists F, G, and H are selected by no query.

Table 3.1. Sizes in the running example and in the real implementation.

	Example	Real implementation
Tile size	4×4	32×32
Queries per batch	4 (q_0 – q_3)	32
Vector dimensions (padded)	3 (4)	100 (128)
Vectors per page	4	32
Inverted lists N_{list}	8 (A–H)	512–2048
Lists per query N_{probe}	2	1–32
Top- k state per query	top-2	top-32
Results returned per query	2	$k \leq 32$ ($k = 10$ evaluated)
Fine-search workers	3 (W1–W3)	63
Chunk limit	2 pages	128 pages

3.2 Device-Oriented IVF Representation

3.2.1 Index Construction

Index training is treated as an offline preprocessing step and is outside the timed search path. The training split is clustered with Faiss k -means, using its default

settings, which sample at most 256 training vectors per centroid, to obtain N_{list} centroids [12], following the same training procedure used by the CPU and GPU baselines based on Faiss `IndexIVFScalarQuantizer` with `QT_fp16` storage. The trained centroid matrix is exported and reused by the Tenstorrent implementation. Reusing the same centroids avoids making index-training quality an additional experimental variable and allows the evaluation to focus on differences in search execution.

Training is not performed on the Tenstorrent device because the objective of this work is the online IVF search path rather than accelerator-based k -means. An on-device training implementation would require additional iterative kernels for vector assignment, per-cluster accumulation, centroid normalization, and convergence control, together with temporary device-memory capacity for the intermediate state. Those operations would neither be part of the measured query path nor improve the fairness of the search comparison. During construction of the Tenstorrent-oriented index, each database vector is assigned to its closest exported centroid and placed in the corresponding packed inverted list.

The Tenstorrent index stores full, uncompressed database vectors rather than compressed representations. Database vectors and centroids are normalized before assignment and search, allowing inner product to be used as the similarity measure.

3.2.2 Tiled Data Layout and Memory Placement

As introduced in Chapter 2, Tensix computation is organized around 32×32 tiles [28]. Let D denote the original vector dimension. To match this tile-oriented execution model, the dimension is padded to

$$D_p = 32 \left\lceil \frac{D}{32} \right\rceil. \quad (3.2)$$

All values introduced by dimensional padding are set to zero on the host before the data are transferred to the device.

Vectors assigned to the same inverted list are stored consecutively in one logical packed database buffer. For a list containing L_j valid vectors, its stored length is rounded to a multiple of 32:

$$L_{j,p} = 32 \left\lceil \frac{L_j}{32} \right\rceil. \quad (3.3)$$

Each group of 32 vectors forms one *candidate page*. A page contains $D_p/32$ 16-bit floating-point (FP16) data tiles, each of shape 32×32 , which together represent the 32 candidate vectors over the complete padded dimension. Centroids and queries use the same FP16 representation, while vector identifiers are stored as 32-bit unsigned integers (UInt32). The Tenstorrent compute configuration enables FP32 destination accumulation. In this implementation, this is not merely a numerical-precision choice but a compatibility requirement of the employed top- k kernel path, which applies the score ordering to companion UInt32 identifier tiles.

A candidate page must not be confused with a query batch. A query batch contains up to 32 queries and forms the row operand of fine search, whereas a candidate page contains 32 database vectors and forms the column operand. Their

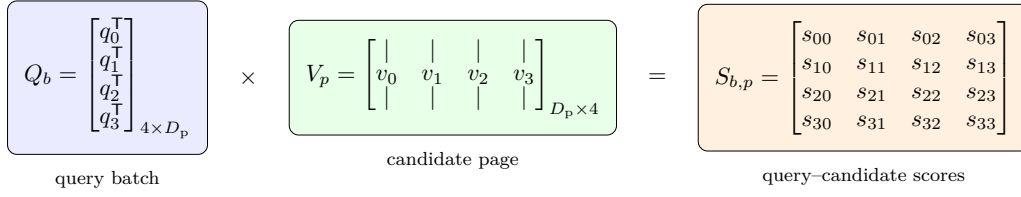


Figure 3.2. 4×4 example of one fine-search matrix multiplication. Rows represent queries and columns represent candidate vectors. The implementation uses 32×32 tiles rather than the reduced size shown here.

matrix product evaluates every query in the batch against every vector in the page. Figure 3.2 illustrates this distinction with the 4×4 tiles of the running example.

For every candidate page, one full 32×32 `UInt32` identifier tile is stored. In its DRAM representation, each column corresponds to one candidate vector and the same 32 identifiers are replicated across all query rows. For candidate identifiers $i_0, \dots, i_{31}$, the stored tile is

$$I^{\text{stored}} = \begin{bmatrix} i_0 & i_1 & \cdots & i_{31} \\ i_0 & i_1 & \cdots & i_{31} \\ \vdots & \vdots & \ddots & \vdots \\ i_0 & i_1 & \cdots & i_{31} \end{bmatrix}. \quad (3.4)$$

Before the compute-kernel `topk_local_sort` operation, both the similarity tile and its companion identifier tile are transposed. The identifier tile presented to the selection operation is therefore

$$I^{\text{topk}} = \left(I^{\text{stored}}\right)^T = \begin{bmatrix} i_0 & i_0 & \cdots & i_0 \\ i_1 & i_1 & \cdots & i_1 \\ \vdots & \vdots & \ddots & \vdots \\ i_{31} & i_{31} & \cdots & i_{31} \end{bmatrix}. \quad (3.5)$$

After transposition, each column corresponds to one query and the rows represent candidate entries. The `topk_local_sort` operation sorts candidate entries independently for each query column. Applying the same transpose and ordering operations to the score and identifier tiles preserves the association between every similarity value and its original database-vector identifier. The same mechanism updates every running top-32 state in coarse and fine search (Section 3.5.1).

Identifiers are stored as complete tiles because data are fetched from DRAM and processed by the compute engines in units of tiles, and `topk_local_sort` requires a full identifier tile next to the score tile. The cost is that a 4,096-byte identifier tile accompanies every 8,192-byte page of vectors, so one third of the bytes read by fine search are identifiers.

The vectors of all inverted lists are stored sequentially in a single packed device buffer. Each inverted list is identified by its offset within this buffer and by its length, rather than being stored in a separate device allocation. Vector identifiers occupy a separate packed `UInt32` buffer using the same list and page ordering. An inverted-list offset therefore identifies a position in this global logical representation.

At the physical memory level, TT-Metalium interleaves buffer pages across DRAM banks, so “contiguous” refers to the logical buffer layout rather than to consecutive bytes within one DRAM bank.

Figure 3.3 shows this layout for the running example. List A holds 7 vectors and therefore occupies two pages, the second with one padded slot; list C holds 10 vectors in three pages and is longer than the example chunk limit of two pages, so it is later scheduled as two tasks (Section 3.4.2).

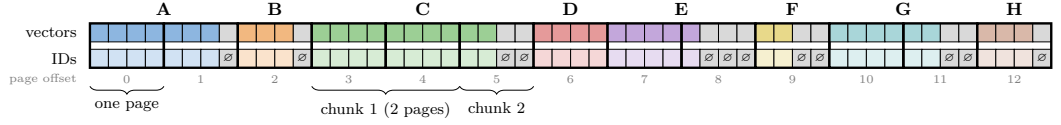


Figure 3.3. Packed layout of the example index in device DRAM. Each thick box is one page of 4 vectors with its identifier tile (one row of it is drawn; the stored tile repeats this row for every query row), and the numbers below give the page offsets. Gray slots are list padding carrying the sentinel identifier $\emptyset$. List C exceeds the chunk limit of 2 pages and is scheduled as two chunks.

Persistent search data are allocated in device DRAM, including the centroid matrix, packed database-vector buffer, and packed vector-identifier buffer. Query buffers and worker descriptions are likewise allocated in DRAM for each search workload, while per-core L1 memory is used as transient kernel-local storage through circular buffers. During execution, data required by the active kernels are streamed from DRAM into L1 rather than persistently cached across search invocations. The exact reuse and streaming strategy of the coarse- and fine-search programs is described in Sections 3.3 and 3.5.1, respectively.

Padding is performed entirely on the host before device transfer:

- database dimensional padding and inverted-list length padding are generated during `IVF_TT::create_index`;
- centroid dimensional padding is generated while constructing the centroid buffer in `load_to_device`;
- query dimensional padding and final-batch row padding are generated in `search_all`.

Padded vector identifiers are assigned the sentinel value `0xffffffff` on the host.

Padded candidate positions contain zero-valued vectors and carry the sentinel identifiers defined above. Because the kernel also knows the size of every list it processes, it recognizes the last page of a list and, if that page is partially populated, linearly scans its score and identifier entries before the top- k update. Whenever a sentinel identifier is encountered, the corresponding similarity value is replaced with -10000 , preventing padded candidates from displacing valid results during selection. The database and query vectors are normalized, so a valid inner product is bounded below by -1 in exact arithmetic; -10000 is therefore safely outside the valid similarity range. Complete candidate pages contain 32 valid vectors and bypass this masking step, avoiding an unnecessary scan of all $32 \times 32 = 1024$ entries when no padding is present.

3.3 Coarse Search

The purpose of coarse search is to identify the inverted lists most relevant to each query. For a valid query batch $Q_b \in \mathbb{R}^{32 \times D_p}$ and centroid matrix $C \in \mathbb{R}^{D_p \times N_{\text{list}}}$, the logical operation is

$$S_b^{\text{coarse}} = Q_b C, \quad (3.6)$$

where each row of S_b^{coarse} contains the similarities between one query and all centroids.

Tiling proceeds over blocks of 32 centroids. After loading the query batch from DRAM into an L1 circular buffer, the reader kernel streams centroid blocks from device DRAM while retaining the query tiles for reuse across the complete centroid matrix. Let $C_c \in \mathbb{R}^{D_p \times 32}$ denote centroid block c .

The corresponding coarse-similarity tile is

$$S_{b,c}^{\text{coarse}} = Q_b C_c, \quad S_{b,c}^{\text{coarse}} \in \mathbb{R}^{32 \times 32}. \quad (3.7)$$

Rather than materializing all N_{list} similarities in DRAM, the compute kernel incrementally maintains the 32 highest centroid scores and their identifiers for each query row. This running top-32 state uses the same transpose-and-topk_local_sort mechanism described in Section 3.2.2, with identical ordering decisions applied to scores and centroid identifiers.

Figure 3.4 shows coarse search for the running example. The query tile is multiplied with the eight centroids in two blocks of four, and after each block the best two lists of every query are updated. After the first block, q_2 holds C (0.9) and D (0.3); in the second block, E scores 0.8 and replaces D, so the final selection of q_2 is {C, E}. The other queries keep their first-block selections, since no list of the second block beats them.

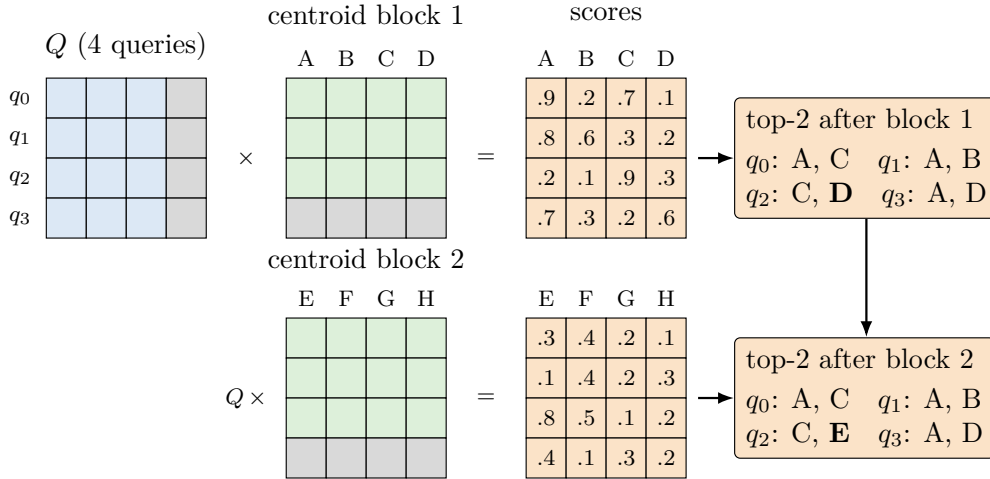


Figure 3.4. Coarse search in the example. The query tile stays in L1 while the centroid blocks are streamed from DRAM; after each block, the best two lists of every query are updated; in the second block, E replaces D for q_2 . Gray cells are dimension padding.

Once all centroid blocks assigned to a query batch have been processed, the coarse-search writer kernel running on the same participating Tensix core writes one score tile and one identifier tile for that batch to device DRAM. After copying

these results to the host, the host retains the first N_{probe} centroid identifiers of each valid query. Because the queries of a batch can select the same lists, the resulting $32 \times N_{\text{probe}}$ matrix of identifiers can contain the same list several times; the host removes these duplicates when it forms the batch-level union described in Section 3.4.1. Because the coarse search keeps a top-32 state per query, the current implementation limits N_{probe} to at most 32.

The coarse-search program distributes the query batches across all available Tensix cores, with each active core processing one or more complete batches and avoiding inter-core communication during the centroid scan. Because every valid query is evaluated against the same centroid matrix, coarse search remains regular and requires a predictable amount of work.

3.4 Fine-Search Scheduling

Fine search introduces the principal source of irregularity in the system. Every valid query selects the same number N_{probe} of inverted lists, but the identifiers and lengths of those lists depend on the query. Two queries with the same N_{probe} may therefore require very different amounts of candidate processing. The imbalance remains after lists are combined at batch level: one worker assigned several short lists can finish before another worker assigned one very long list.

The scheduler estimates the cost of a selected list by its number of padded 32-vector candidate pages. Lists longer than 128 pages, that is 4,096 vectors, are split into consecutive chunks of at most 128 pages, each scheduled as a separate task, while shorter lists remain a single task. The limit of 128 pages was chosen experimentally, as the value that performed best among those tested for the evaluated list sizes.

3.4.1 Batch-Level List Union

After coarse search, each query row has up to N_{probe} selected inverted-list identifiers. Since fine search operates on a complete batch of up to 32 queries, the host collects these identifiers into a single set of lists for the batch.

Let $\mathcal{P}_{b,r}$ denote the N_{probe} lists selected for query row r of batch b . The set of lists processed for the batch is

$$\mathcal{U}_b = \bigcup_{r \in \mathcal{V}_b} \mathcal{P}_{b,r}, \quad (3.8)$$

where $\mathcal{V}_b$ contains the valid query rows of the batch.

Operationally, the host iterates over the coarse-search results of every valid query row and inserts the first N_{probe} list identifiers into a set. Duplicate identifiers are therefore automatically removed. The resulting set $\mathcal{U}_b$ contains every inverted list selected by at least one query in the batch.

The batch-level union follows from the execution granularity of the Tensix matrix engine. Multiplying a page $V_p \in \mathbb{R}^{D_p \times 32}$ by the query tiles $Q_b \in \mathbb{R}^{32 \times D_p}$ costs the same whether one or all 32 query rows need its scores, so once a page has been fetched, scoring it for every query of the batch requires no additional multiplication.

As a consequence, a query can also obtain similarity scores for vectors belonging to a list that was selected by another query in the same batch. The effective candidate set for one query can therefore be larger than the set defined by its own N_{probe} lists. This is an implementation consequence of processing the batch-level union and does not modify the stored IVF index. Section 5.2 shows that, with queries batched in dataset order, the queries of a batch share few lists, so the union is close to $32N_{\text{probe}}$ lists and each query is scored against 12 to 32 times as many vectors as its own lists contain.

The union can nevertheless increase the total amount of fine-search work because lists selected by different queries may not overlap, increasing the number of distinct candidate pages that must be processed. The benefit of the design therefore comes from fully exploiting each tile-level matrix multiplication, rather than from making the additional lists computationally free.

3.4.2 Greedy Work Distribution

For each list $j \in \mathcal{U}_b$, the host creates one task for the entire list or one task per chunk, as described above. A list j with L_j valid vectors has

$$n_j = \left\lceil \frac{L_j}{32} \right\rceil \quad (3.9)$$

padded pages; lists containing 320 and 288 vectors, for example, have 10 and 9 pages. The weight n_t of a task t is its own page count, which equals n_j for a complete list and at most 128 for a chunk, and counts the fixed-shape page multiplications the task requires.

The fine-search program uses all Tensix cores available to it, an 8×8 logical grid of 64 cores. Logical core $(0,0)$ is reserved for final aggregation, and the remaining 63 cores form the fine-search worker set $\mathcal{W}$. For every query batch, let $\mathcal{A}_{b,w}$ denote the tasks assigned to worker w . The scheduler resets both the assignments and their estimated loads,

$$\mathcal{A}_{b,w} = \emptyset, \quad \lambda_{b,w} = 0, \quad w \in \mathcal{W}, \quad (3.10)$$

sorts the tasks in non-increasing order of n_t , and assigns each task t to a currently least-loaded worker:

$$w^* = \arg \min_{w \in \mathcal{W}} \lambda_{b,w}, \quad (3.11)$$

$$\mathcal{A}_{b,w^*} \leftarrow \mathcal{A}_{b,w^*} \cup \{t\}, \quad (3.12)$$

$$\lambda_{b,w^*} \leftarrow \lambda_{b,w^*} + n_t. \quad (3.13)$$

Tasks with equal page counts are ordered by ascending list identifier, and chunks of the same list by their position in the list. If several workers have the same minimum load, the worker with the lowest index is selected. These explicit tie rules make the host schedule reproducible. The resulting longest-processing-time-first (LPT) greedy policy balances the padded page count of each batch while retaining sequential access within every packed list. Because the loads are reset for every batch, a batch with fewer tasks than workers always assigns its work to the lowest-indexed workers, and imbalances are not compensated across batches; Section 5.3.1 shows how worker imbalance sets the batch completion time.

Figure 3.5 shows the complete process for the running example. The four queries select eight lists, but only five are distinct, since A is shared by three queries and C by two, so $\mathcal{U}_b = \{A, B, C, D, E\}$. List C has three pages, more than the example chunk limit of two, so it becomes two tasks, C_1 with two pages and C_2 with one. Sorted by weight and assigned to the least-loaded worker, the six tasks give each of the three workers exactly three pages. Every worker then scores all four queries against its pages, so query q_1 , for example, also receives scores for list C, which it did not select.

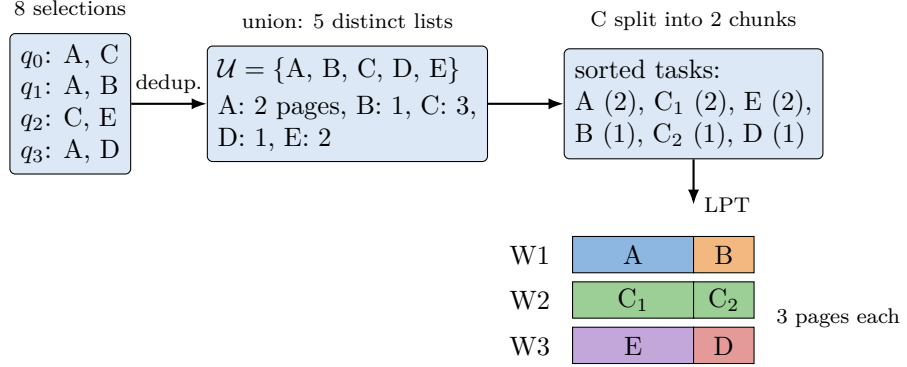


Figure 3.5. From selections to worker tasks in the example. The union removes duplicate lists, the long list C is split into chunks, and the tasks are assigned longest-first to the least-loaded worker (LPT).

The page count is an estimate rather than a cycle-accurate cost model. It does not capture per-list setup, masking of the final partial page, variation in memory service time, circular-buffer waiting, or synchronization. Moreover, because tasks remain coarse, with up to 128 pages each, the largest task can still determine the batch completion time. After scheduling, the host serializes one description per fine-search worker so that its reader kernel can locate the assigned lists and chunks in device memory.

3.4.3 Worker Description

The host serializes the assignment of each fine-search worker into a flattened worker description in DRAM, which may span several DRAM pages. For B query batches (Section 3.1.1), the description of worker w is

$$\mathcal{D}_w = \langle B, R_{w,0}, R_{w,1}, \dots, R_{w,B-1} \rangle, \quad (3.14)$$

with one record per batch,

$$R_{w,b} = \langle o_b^Q, n_b^Q, N_{w,b}, E_{w,b,0}, \dots, E_{w,b,N_{w,b}-1} \rangle, \quad E_{w,b,e} = \langle o_{w,b,e}^V, n_{w,b,e}^V \rangle. \quad (3.15)$$

Here o_b^Q and n_b^Q are the offset and tile count of query batch b in the query buffer, and $N_{w,b}$ is the number of tasks assigned to worker w for that batch. Each entry $E_{w,b,e}$ describes one task, a complete list or a chunk, by its page offset $o_{w,b,e}^V$ in the packed database buffer and its page count $n_{w,b,e}^V = n_t$. Because every page holds

32 vectors, the offset and page count are enough for the reader to fetch the task’s vector and identifier tiles. In the running example, where the pages of lists A to H occupy the packed offsets 0–1, 2, 3–5, 6, 7–8, 9, 10–11, and 12 (Figure 3.3), the single batch record of each worker is

W1: $\langle 0, 1, 2, \langle 0, 2 \rangle, \langle 2, 1 \rangle \rangle$ W2: $\langle 0, 1, 2, \langle 3, 2 \rangle, \langle 5, 1 \rangle \rangle$ W3: $\langle 0, 1, 2, \langle 7, 2 \rangle, \langle 6, 1 \rangle \rangle$,

that is, query offset 0, one query tile, two tasks, and one offset–count pair per task.

At the start of fine search, each worker copies its description into L1. For every batch with assigned work, the reader loads the query tiles once into the query circular buffer, where they stay while all tasks of that batch are processed, and then moves to the next batch record.

3.5 Fine Search and Result Reduction

3.5.1 Worker Pipeline

Each of the 63 fine-search workers executes reader, compute, and writer kernels concurrently, working through its description as described in Section 3.4.3. Figure 3.6 shows where data move during the search and how the RISC-V cores of one Tensix core share the work of a worker: the NCRISC core runs the reader, the three TRISC cores run the unpack, math, and pack stages of the compute kernel, and the BRISC core runs the writer.

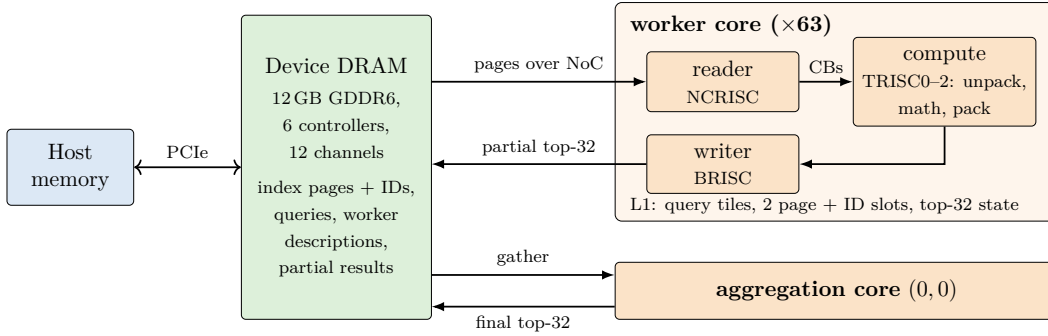


Figure 3.6. Data movement during the search and roles of the RISC-V cores of a worker.

The index is uploaded once; for each workload, the queries are written to device DRAM, and every worker streams its pages over the network-on-chip (NoC) into L1 circular buffers (CBs). The reader, compute, and writer kernels run on different RISC-V cores, so the next page can be read while the current one is scored. The partial results reach the aggregation core through DRAM.

Candidate vectors and their identifier tiles are streamed from the packed DRAM buffers. The reader issues both reads before a shared synchronization barrier, allowing their transfers over the on-device Network-on-Chip (NoC) to be outstanding concurrently. Double-buffering of the candidate circular buffers allows the reader to prepare a later page while the compute kernel operates on an earlier one, subject to circular-buffer capacity and producer–consumer synchronization.

The candidate-data and candidate-identifier circular buffers provide two alternating slot pairs. After the reader fills slot pair 0 with page p , the compute kernel

can consume that page while the reader fills slot pair 1 with page $p + 1$. Once compute releases slot pair 0, the reader can reuse it for page $p + 2$ while compute consumes slot pair 1. This pipelining can hide part of the page-read cost, although the achieved overlap remains subject to NoC completion, circular-buffer availability, and relative reader and compute progress.

Let t identify a task assigned to the worker and p a page within it. For one such page $V_{t,p} \in \mathbb{R}^{D_p \times 32}$, the compute kernel evaluates

$$S_{b,t,p}^{\text{fine}} = Q_b V_{t,p}, \quad (3.16)$$

producing a 32×32 tile of query–candidate similarities. Figure 3.7 follows one page through the compute kernel in the running example: worker W1 multiplies the query tile with the second page of list A, whose last slot is padding, masks the padded column through its sentinel identifier, and merges the scores into its running top-2.

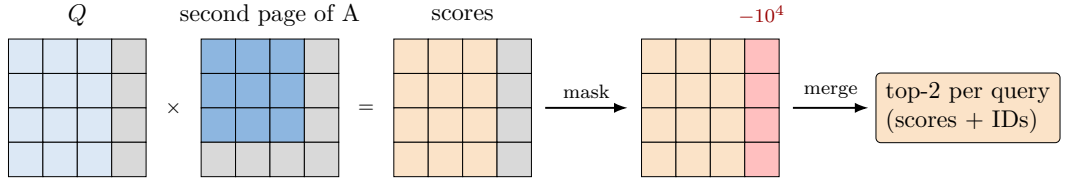


Figure 3.7. One page in the example. Columns of the page are vectors; its last row is dimension padding and its last column a padded vector slot (gray). The padded slot is recognized through its sentinel identifier, and its score is set to -10000 before the merge into the running top-2.

Across all candidate pages assigned to it, the worker incrementally maintains a row-wise top-32 state, with scores and original vector identifiers propagated together through the same ordering operations. No list-local to global identifier reconstruction is therefore required after the search.

Figure 3.8 shows the local top-32 update performed by one fine-search worker. Two 32×32 tiles hold the best scores and identifiers found so far, while the other two contain the scores and identifiers of the current candidate page. As described in Section 3.2.2, before selection the score and identifier tiles of the current page are transposed so that each column corresponds to one query and the rows contain candidate entries; the saved state is already kept in this transposed orientation, so it needs no further transposition. The `topk_local_sort` operation then orders, for every query column, the entries of the saved state together with those of the current page, keeps the best 32, and applies the same decisions to the identifier tiles.

3.5.2 Hierarchical Top- k

A single worker observes only part of the candidate set, so the final result is computed hierarchically. For query row r in batch b , worker w first retains

$$T_{b,w,r} = \text{Top}_{32}(\mathcal{C}_{b,w,r}), \quad (3.17)$$

where $\mathcal{C}_{b,w,r}$ is the set of candidates evaluated by that worker.

Worker-local results are staged in DRAM. Each worker owns one fixed slot in a partial-result buffer, holding one score tile and one identifier tile, and writes the

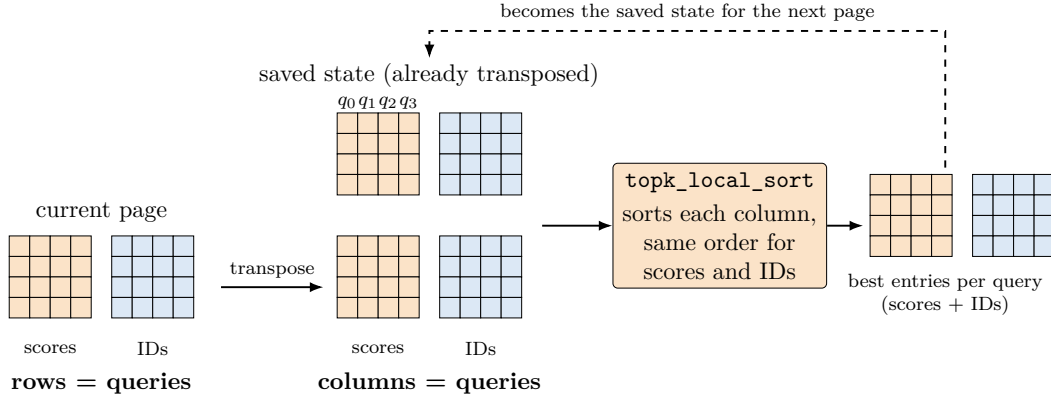


Figure 3.8. Worker-local top-32 update, drawn with 4×4 tiles. The score and identifier tiles of the current page are transposed so that each column corresponds to one query, the orientation in which the saved state is already kept; `topk_local_sort` then orders the saved and current entries of every column, keeps the best ones, and applies the same order to scores and identifiers. The result becomes the saved state for the next page. The implementation uses 32×32 tiles and keeps the best 32 entries.

result of every batch to this same slot. All configured workers produce and signal one partial result for every batch, even when a worker has no active list task for that batch. In that case, every score remains at the initial value -10000 and every identifier remains at `0xffffffff`.

Synchronization uses a cumulative completion semaphore in the L1 memory of logical core (0,0). After writing its partial result for a batch, each worker increments this semaphore, and the aggregation reader waits until the expected number of increments has been observed. Once the partial result of a worker has been consumed, the aggregation reader increments a worker-specific ready semaphore in that worker’s L1 memory. Before writing the result of the next batch, the writer kernel of the worker waits on this semaphore, so that it never overwrites a result that core (0,0) has not yet read; the reader and compute kernels of the worker meanwhile continue with the next batch.

Figure 3.9 illustrates this two-way handshake. Workers write their local top-32 results to their own slots and signal completion to the aggregation core; after reading all slots, core (0,0) signals each worker that its slot can be overwritten.

The aggregation compute kernel performs the second reduction,

$$T_{b,r} = \text{Top}_{32} \left(\bigcup_{w \in \mathcal{W}} T_{b,w,r} \right), \quad (3.18)$$

and writes one final score tile and one final identifier tile per batch to DRAM. Figure 3.10 shows the two-level selection for the running example: each of the three workers holds the best two candidates per query among its own pages, and the aggregation core merges these 3×2 candidates into the final top-2.

The final top-32 is therefore computed on the device, and only this result is transferred to the host. The host converts the tiled outputs to row-major form, removes sentinel entries, discards padded query rows, and retains the first k results

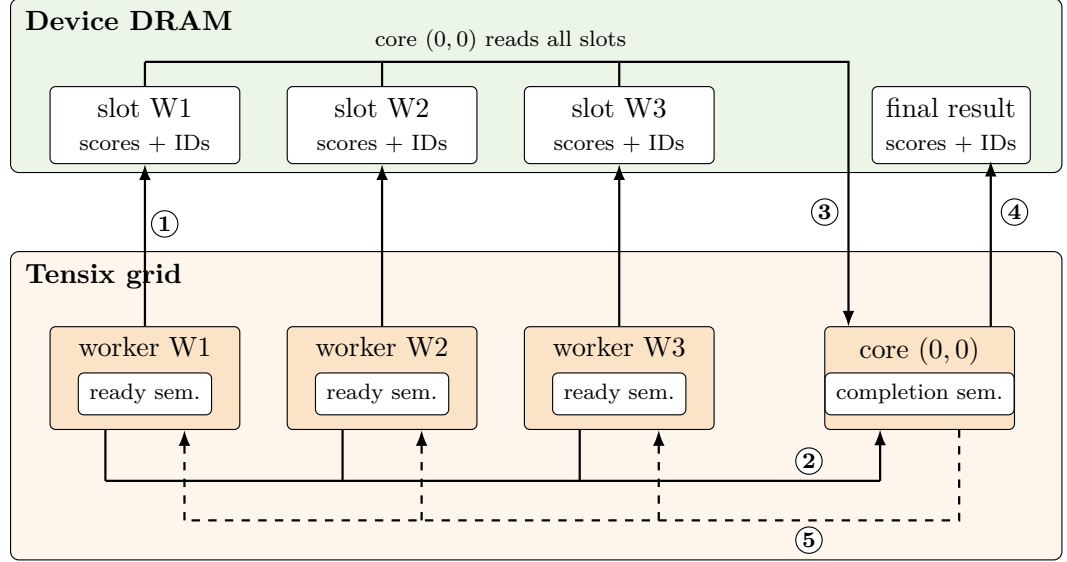


Figure 3.9. DRAM staging and synchronization of partial results for one batch, shown for three workers (63 in the implementation). (1) Each worker writes its local top-32 to its own DRAM slot and (2) increments the completion semaphore of core (0,0). (3) Once all workers have signaled, core (0,0) reads the slots, merges them, and (4) writes the final result to DRAM. (5) It then increments the ready semaphore of each worker, whose writer waits for this signal before writing the next batch to the same slot.

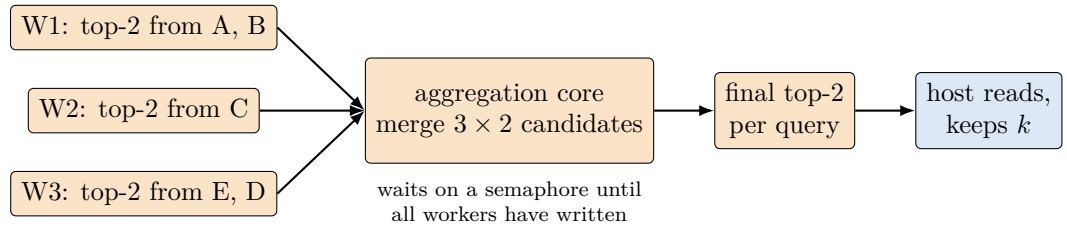


Figure 3.10. Hierarchical top- k in the example. In the real implementation, 63 workers each contribute a top-32 per query, and the aggregation core produces the final top-32 on the device.

for every valid query. The implementation therefore supports $k \leq 32$; the evaluation in this thesis uses $k = 10$.

For B query batches and W fine-search workers, logical core $(0, 0)$ consumes BW worker-result pairs and performs BW running top-32 updates. Synchronization activity and partial-result traffic also grow linearly with the number of worker results. With $W = 63$, each batch contributes 63 score-and-identifier result pairs to the aggregation stage and requires 63 running top-32 updates. Because workers without active list tasks still write sentinel results, this cost depends on the configured worker count rather than the number of non-empty assignments. The centralized stage consequently scales as $\Theta(BW)$ and can constrain designs with larger worker counts.

3.6 Multi-Batch Execution

The first implementation processed one batch of at most 32 queries per search invocation. This design matched the 32×32 tile representation and simplified the initial implementation, but a larger query workload required the host to repeat query preparation, coarse execution, synchronization, list selection, fine-search scheduling, fine execution, final result transfer, and host-side orchestration for every group of 32 queries.

The final implementation, represented by `ann_ivf_dram_args`, retains the same 32-query internal representation while allowing one search call to process multiple batches. After a coarse-search invocation processes all valid batches of the workload, the host constructs the fine-search descriptions for the complete set of selected lists. These logical descriptions may occupy multiple DRAM pages, so the format does not impose a fixed batch-count limit. The fine-search program then processes the paged descriptions within one larger invocation; practical capacity remains bounded by the device-memory allocation available to the workload. Within this invocation, each worker moves to its tasks of the next batch as soon as it has finished the current one; only its writer waits, if necessary, until the previous result has been read from its slot. The aggregation core reduces a batch as soon as all workers have written it, while the workers already process later batches (Figure 3.11).

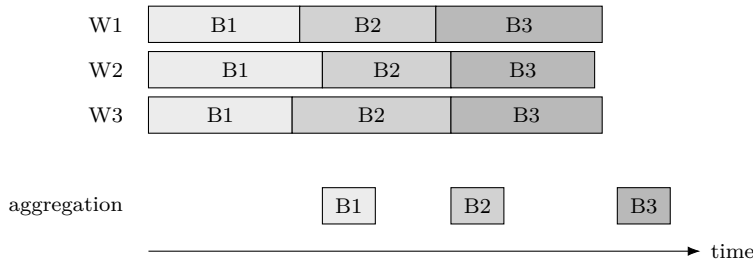


Figure 3.11. Three batches B1–B3 in one invocation. Workers do not wait for each other between batches; the aggregation core reduces a batch once the last worker has finished it, while the workers already process later batches.

This change separates tile size from execution granularity: a batch of 32 queries remains the natural unit of matrix computation, but it is no longer the unit at which the complete device pipeline must be launched. Repeated program configuration,

launch, synchronization, and host orchestration are consequently amortized across a larger number of queries.

3.7 Implementation Constraints

Table 3.2 consolidates the implementation constraints derived in the preceding sections without repeating their full derivations.

Table 3.2. Principal constraints of the evaluated Tenstorrent implementation.

Constraint	Cause	Consequence
Tile alignment	The compute path operates on 32×32 tiles.	Vector dimensions and list lengths are padded to multiples of 32; an internal query batch contains up to 32 valid queries.
Centroid blocking	Coarse search traverses complete 32-centroid blocks.	The current representation requires N_{list} to be divisible by 32.
Selection capacity	Coarse, worker-local, and global selection use top-32 states.	$N_{\text{probe}} \leq 32$ and $k \leq 32$. The reported experiments use $k = 10$.
Task granularity	A worker record addresses one list or one chunk of at most 128 pages.	Tasks remain coarse, so a large task can determine the batch tail even after greedy balancing.
Paged worker descriptions	A logical description may span multiple DRAM pages.	The format imposes no fixed batch-count limit; practical capacity depends on the device-memory allocation available to the invocation.
Centralized reduction	Every worker emits one partial result for every batch.	Aggregation work and intermediate traffic scale as $\Theta(BW)$, including sentinel results from workers with empty assignments.
Deployment scope	The implementation targets one Wormhole n150 device.	Multi-device approximate nearest-neighbor execution is outside the scope of this work.

Reader, compute, and writer kernels additionally share the local L1 capacity through their circular buffers, query tiles, intermediate similarities, identifier tiles, running top-32 states, and worker metadata. Buffer depth must therefore balance query reuse and double buffering against local-memory capacity. Progress also remains subject to circular-buffer backpressure, NoC completion, and the synchronization dependencies described above.

3.8 Chapter Summary

This chapter described the implementation of IVF-Flat search on a single-chip Tenstorrent Wormhole n150 device, with the index transformed into a tile-oriented

representation in which vectors are grouped by inverted list and stored in 32-vector pages together with their original identifiers.

The search pipeline is divided into a regular coarse-search phase and an irregular fine-search phase. While coarse search reuses query tiles to scan the centroid matrix and return a fixed-size set of centroid candidates, the host subsequently selects the N_{probe} lists, forms batch-level unions, and assigns the selected lists, or chunks of long lists, to workers using their padded page counts as weights in a greedy longest-task-first scheduler.

Each fine-search worker reuses one query batch while streaming the candidate pages of its assigned lists and maintaining a worker-local row-wise top-32 state. The resulting partial results are staged in DRAM and merged on logical core (0, 0) before the final outputs are returned to the host.

The final design preserves the natural 32-query tile representation while extending execution to multiple batches per invocation. This reduces repeated host-device orchestration while preserving the same coarse- and fine-search stages and using batch-level list unions to match the tile-oriented execution model. The following chapter defines the methodology used to evaluate the resulting implementation.

Chapter 4

Experimental Methodology

This chapter describes how the two host-CPU configurations, the NVIDIA A100 system, and the Tenstorrent implementation are compared in terms of throughput, search quality, and energy. Because the Tenstorrent implementation described in Chapter 3 does not associate N_{probe} with the same candidate set as conventional per-query IVF search, cross-platform comparisons are made at matched measured recall rather than at matched index parameters.

4.1 Evaluation Goals

The experiments address the following evaluation goals:

1. compare throughput as a function of measured Recall@10 rather than at one isolated search setting;
2. determine how N_{list} and N_{probe} affect the trade-off between throughput and recall;
3. identify the dominant portions of the Tenstorrent search path and inspect overlap and imbalance within its fine-search program;
4. compare the energy of a complete query workload at approximately matched recall.

A higher-throughput operating point at lower recall is never treated as preferable to a slower point at higher recall unless the quality difference is made explicit.

4.2 Hardware and Software Configuration

Table 4.1 summarizes the four evaluated system configurations. The CPU and GPU baselines use Faiss `IndexIVFScalarQuantizer` with `QT_fp16` storage [12], while the Tenstorrent system uses the custom `ann_ivf_dram_args` program presented in Chapter 3, implemented with TT-Metalium [28]. Both therefore store full-dimensional database vectors in FP16. The CPU and GPU runs used nodes of the Leonardo cluster [31], and the Tenstorrent runs used `node117` of the Computer Science Department cluster of Sapienza University.

Table 4.1. Evaluated ANN search configurations. Accelerator configurations include the host required to execute and orchestrate the search.

Configuration	Evaluated hardware	Allocation and implementation
CPU (56 cores)	Intel Xeon Platinum 8480+ node with 112 physical cores	56 physical cores, Faiss IVF-SQ (FP16)
CPU (112 cores)	Intel Xeon Platinum 8480+ node with 112 physical cores	112 physical cores, Faiss IVF-SQ (FP16)
GPU (A100) + CPU	NVIDIA A100-SXM 64 GB; Intel Xeon Platinum 8358 host	One GPU and one allocated host core, Faiss GPU IVF-SQ (FP16)
TT (Wormhole n150) + CPU	Single-chip Wormhole n150; AMD EPYC 7301 host	Custom TT-Metalium IVF with FP16 lists, host-side list selection and scheduling

The CPU node exposes 112 physical cores in eight NUMA domains, with one hardware thread per core. In the 56-core configuration, Slurm exposed 56 CPU identifiers and Faiss reported the same number of execution threads. All CPU and GPU energy sessions ran on nodes allocated exclusively, so no other job shared the node during a measurement. The GPU node combines NVIDIA A100-SXM 64 GB accelerators, with a power limit of 550 W as reported by `nvidia-smi`, with a 32-core Intel Xeon Platinum 8358 host in two NUMA domains. The CPU and GPU baselines used the same Python 3.11.7 environment with Faiss 1.14.1 and GCC 12.2.0, and the GPU runs additionally used CUDA 12.2 and NVIDIA driver 535.274.02. The Tenstorrent host reports 64 online logical CPUs with two hardware threads per core, eight NUMA domains, and 251 GiB of memory, and the experiments used TT-SMI 3.0.38 and TT-Metalium v0.66.0. The host processors differ considerably in their rated thermal design power (TDP): each Xeon Platinum 8480+ is rated at 350 W, about 700 W for the dual-socket CPU node, the single Xeon Platinum 8358 of the GPU node at 250 W, and each EPYC 7301 at 155–170 W, about 310–340 W for the dual-socket Tenstorrent host.

The accelerators are not evaluated as stand-alone devices, because data preparation, invocation, synchronization, and result handling all require host work. The A100 variant is stated explicitly because PCIe and SXM models differ in memory and power characteristics [18]; the Wormhole n150 follows Tenstorrent’s board specifications [30].

4.3 Dataset and Index Parameters

All systems search the ANN-Benchmarks `glove-100-angular` dataset [2, 3], built from the 100-dimensional Stanford GloVe Twitter vectors [22] with a fixed train/test split. It contains 1,183,514 database vectors and 10,000 test queries, together with the 100 exact nearest neighbors of every query under angular distance. Database

vectors, centroids, and queries are normalized so that angular similarity reduces to an inner product.

For each N_{list} , centroids are trained once with Faiss k -means on the training split and saved to file. Faiss’s default settings are used, which sample at most 256 training vectors per centroid. Every implementation loads the same centroid matrix and assigns the database vectors to inverted lists itself, so clustering quality is not an experimental variable. On Tenstorrent, vectors, queries, centroids, and score tiles use FP16, identifiers use UInt32, and FP32 destination accumulation is enabled because the top- k path requires it (Section 3.2.2). The compute kernels use the default TT-Metalium math fidelity, HiFi4. Because the multiplier of the Tensix matrix engine is narrower than an FP16 mantissa, each product is computed in one to four passes over parts of the mantissas: the lower fidelities, LoFi to HiFi3, use fewer passes and drop the least significant bits to run faster, whereas HiFi4 uses all four and multiplies the complete mantissas. The similarity scores are therefore computed at the full precision of the FP16 inputs, which matters in nearest-neighbor search, where close candidates can differ only in the last bits of their scores.

All four configurations are evaluated on the grid

$$N_{\text{list}} \in \{512, 1024, 2048\}, \quad N_{\text{probe}} \in \{1, 2, 4, 8, 16, 32\}. \quad (4.1)$$

The Faiss CPU and GPU runs add $(N_{\text{list}}, N_{\text{probe}}) = (512, 64)$ to reach the recall of Tenstorrent at $(512, 32)$, and Tenstorrent adds $(2048, 12)$ as a recall-matching point for the energy comparison. Because the Tenstorrent implementation maintains top-32 states, it supports $N_{\text{probe}} \leq 32$ and $k \leq 32$. All quality results use $k = 10$, matching the public ANN-Benchmarks configuration for this dataset [2, 3].

Changing N_{list} affects both IVF stages: more lists increase the coarse centroid scan and reduce the mean inverted-list size

$$\bar{L} = \frac{N}{N_{\text{list}}}, \quad (4.2)$$

which falls by a factor of four from $N_{\text{list}} = 512$ to 2048, although the work for a given query still depends on the non-uniform sizes of the lists actually selected.

For Faiss, N_{probe} is the number of lists searched for each query. Tenstorrent instead searches, for every query in a batch of up to 32, the union of the lists selected by all queries in that batch (Equation 3.8). A query can therefore be scored against lists selected by other queries, so equal N_{probe} values do not imply equal candidate sets or recall.

4.4 Recall, Throughput, and Matching

Recall@ k follows Equation 2.5 and is reported as the mean over the 10,000 queries; throughout the evaluation, “recall” means Recall@10. Recall is computed in separate untimed quality runs, so ground-truth comparison never inflates search time or energy.

The principal performance metric is queries per second (QPS). If repetition r of $Q_{\text{rep}} = 10,000$ queries takes t_r , then

$$\text{QPS}_r = \frac{Q_{\text{rep}}}{t_r}, \quad \text{QPS}_{\text{reported}} = \text{median}_{r=1,\dots,10} (\text{QPS}_r). \quad (4.3)$$

CPU and GPU results can be compared at common $(N_{\text{list}}, N_{\text{probe}})$, whereas comparisons involving Tenstorrent use measured recall. The energy comparison uses four operating points, grouped around nominal targets of 60%, 78%, 88%, and 95% (Table 4.2); these labels denote approximate quality matching, not uncertainty intervals. The Faiss configurations are shared by the CPU and GPU. Because both use the same centroids and the same per-query list selection, their recall is almost identical: it differs by at most 0.01 percentage points at these four points and by at most 0.04 across the whole grid, that is, by at most about 10 and 40 of the 100,000 true neighbors of the 10,000 queries. The remaining difference arises because the two implementations compute distances with a different order of floating-point operations and select the best candidates with different algorithms, so near-ties between centroids or candidates are occasionally resolved differently. The largest mismatch is at the 60% target, where Tenstorrent is 1.78 percentage points above Faiss, because it was the closest Tenstorrent configuration available.

Table 4.2. Operating points selected for the recall-matched comparisons, written as $(N_{\text{list}}, N_{\text{probe}})$. Faiss recall is given for the CPU and GPU runs respectively.

Target	Faiss config.	Faiss Recall@10 (CPU / GPU)	TT config.	TT Recall@10
60%	(2048, 4)	60.46 / 60.47%	(2048, 4)	62.24%
78%	(2048, 16)	78.33 / 78.33%	(2048, 12)	79.16%
88%	(1024, 32)	88.49 / 88.49%	(1024, 16)	88.03%
95%	(512, 64)	95.54 / 95.53%	(512, 32)	95.37%

4.5 Benchmark Procedure

Throughput runs. Each configuration executes ten timed repetitions of 10,000 queries within one process. For Faiss, the index is initialized once and the ten `index.search` calls reuse the same index, query matrix, and, on the GPU, the same CUDA context; each call receives all 10,000 queries at once. The Faiss runs use no dedicated warm-up call, but the per-run values show no systematic first-repetition slowdown, and the median is insensitive to a single outlier. The Tenstorrent stage and device-profiler experiments include a preceding warm-up invocation, which is excluded from the reported medians and trace. Search timing excludes index construction and the one-time index upload, but includes query-dependent preparation, execution, synchronization, and final result transfer (Section 4.6).

On Tenstorrent, each 10,000-query repetition is split into

$$B_{10k} = \left\lceil \frac{10,000}{32} \right\rceil = 313 \quad (4.4)$$

internal batches of at most 32 queries; the last batch holds 16 valid queries and 16 padded rows, which generate no fine-search tasks. Queries keep their dataset order and are grouped consecutively without shuffling, so the batch unions are reproducible; sensitivity to other groupings is not evaluated.

Run-to-run stability is checked with the relative standard deviation

$$\text{RSD}_{\text{QPS}} = 100 \frac{s_{\text{QPS}}}{\overline{\text{QPS}}}, \quad (4.5)$$

where s_{QPS} and $\overline{\text{QPS}}$ are the sample standard deviation and mean of the ten repetitions.

Energy sessions. Energy is measured on a complete workload rather than on the search calls alone. One session starts a new process, loads the dataset and the saved centroids, assigns the database vectors to inverted lists, transfers the index to the accelerator where applicable, and then executes ten repetitions of 10,000 queries,

$$Q_{\text{run}} = 10 \times 10,000 = 100,000 \quad (4.6)$$

searches in total, before terminating. k -means training and recall evaluation are excluded. Each operating point is measured in ten independent sessions, and the reported energy is the median across them. The spread across sessions is reported as the interquartile range and as the RSD, defined as in Equation 4.5 with session energy in place of QPS. Session time is likewise the median over the ten sessions, and average power is the median energy divided by the median session time. On the CPU node and on the Tenstorrent host, the RAPL counters of both sockets are summed.

This workload boundary deliberately charges initialization to the query workload: it represents the cost of answering 100,000 queries starting from a cold process. Platforms whose search is fast relative to their initialization therefore spend most of the session outside the search calls. Section 5.4 reports this share explicitly.

4.6 Tenstorrent Stage and Device Profiling

The Tenstorrent search path is partitioned into six disjoint host-observed intervals, all measured with `std::chrono::steady_clock` over ten repetitions of 10,000 queries after one warm-up invocation, and reported as per-stage medians:

$$\begin{aligned} T_{\text{search}} = & T_{\text{H2D}} + T_{\text{CPU,coarse}} + T_{\text{TT,coarse}} \\ & + T_{\text{CPU,fine}} + T_{\text{TT,fine}} + T_{\text{CPU,output}}. \end{aligned} \quad (4.7)$$

- T_{H2D} : blocking transfer of query and fine-worker configuration data to the device, excluding the one-time index upload.
- $T_{\text{CPU,coarse}}$: host query preparation, buffer and program setup, and coarse-search runtime configuration.
- $T_{\text{TT,coarse}}$: coarse-workload enqueue through `distributed::Finish()`.
- $T_{\text{CPU,fine}}$: coarse-result handling, batch-level list-union construction, worker scheduling, and fine-program configuration.
- $T_{\text{TT,fine}}$: fine-workload enqueue through completion, including worker execution and final device-side sorting.

- $T_{\text{CPU,output}}$: final device-to-host transfers, utilization, and construction of the returned scores and identifiers.

The two TT intervals include device compute, DRAM and NoC traffic, and synchronization, so they describe complete accelerator stages rather than isolated compute or memory costs.

To relate fine-search cost to the data scanned, a separate untimed run logs, for every internal batch and each of the 19 Tenstorrent configurations, the lists in the batch union $\mathcal{U}_b$, their pages, and the pages assigned to each of the 63 workers. It uses the lists selected by the device coarse search, and one repetition of 10,000 queries suffices because the batching is deterministic; a repeated run at (512, 8) produced identical unions. From this log, the scanned fraction is the number of vectors in the union divided by the 1,183,514 database vectors, and it is compared with the vectors in the query’s own N_{probe} lists, which is what Faiss scans. Worker imbalance is the largest per-worker page load of a batch divided by the mean over all 63 workers. The log treats every list as a single task, whereas the timed runs split lists longer than 128 pages into chunks, which occurs only with 512 lists (Section 3.4.2), so its imbalance values are upper bounds for the timed runs.

Two device-profiler captures, both taken after the warm-up, examine the fine search. The profiler reports device coordinates: profiler core (1, 1) is program-logical core (0, 0), reserved for final aggregation, and profiler cores (1, 2) to (1, 4) are three consecutive fine-search workers. The first capture, at $(N_{\text{list}}, N_{\text{probe}}) = (512, 8)$, shows the last three batches of the measured invocation, with every RISC processor of these four cores on a separate row; each worker zone covers one list task. The second, at (512, 1), zooms into 16 consecutive pages of a single list on one worker, with reader zones for the query fetch and each page’s DRAM fetch, and compute zones for each page. Cycle counts are shifted to a common origin,

$$C_{\text{relative}} = C - C_0, \quad (4.8)$$

where C_0 is the earliest displayed timestamp, and converted to microseconds at the device frequency of about 1 GHz. Because reader, compute, and writer kernels communicate through circular buffers, their zones can include backpressure, input waits, and semaphore synchronization. The trace is therefore used to study overlap and per-core imbalance, not to estimate raw DRAM latency or bandwidth.

4.7 Power and Energy Measurement

CPU energy is read from cumulative RAPL counters. For the accelerator systems, device power is sampled through `nvidia-smi` or `tt-smi` and host energy through RAPL; the two contributions are integrated separately and added. Although power is sampled rather than measured continuously, the sampling is not expected to introduce a significant error. RAPL counters are cumulative, so the energy between two readings does not depend on the sampling interval, and each device trace collects enough samples to follow the power profile: about 20–40 per session for the A100, which is read roughly every 0.13 s, and about 19–35 for the Wormhole, which is read roughly every 0.73 s. Since power stays nearly constant within each phase of a

session, such as idle, initialization, and search, the trapezoidal integration tracks it closely, and taking the median over ten sessions reduces whatever sampling error remains.

On every host, RAPL energy is the sum of one top-level package counter (`package-*`) per socket. Their sub-domains, such as the core domain, are not added, so no energy is counted twice. The package domain covers the processor chip, including its cores, caches, and memory controller, but not the DRAM modules, power supply, fans, or storage, so the host values are a lower bound on the energy of the whole node. Although the Intel hosts also expose a separate RAPL domain for the DRAM modules, it is deliberately left out so that the same package-only boundary is used on all three systems. The AMD EPYC host of the Tenstorrent system provides no DRAM domain. It lists one package domain per die, eight in total over its two sockets, but the four domains of a socket report the same socket-wide counter, so its host energy is read from one domain per socket to avoid counting it four times.

CPU. The Slurm job enables energy profiling, and a background sampler reads the cumulative `energy_uj` counters exposed through `/sys/class/powercap` at a nominal interval of 0.1 s. For an aggregate cumulative reading $R_{\text{RAPL}}(t)$ in micro-joules,

$$E_{\text{CPU}} = 10^{-6} (R_{\text{RAPL}}(t_{\text{end}}) - R_{\text{RAPL}}(t_{\text{start}})). \quad (4.9)$$

Slurm affinity and the number of cores visible to Faiss were checked for each allocation.

GPU. One background stream samples the A100 board power from `nvidia-smi` and another samples the host’s cumulative RAPL energy, with requested sleeps of 50 ms and 100 ms. Command execution makes the effective cadence slower, so the recorded timestamps, not the requested periods, are used. The GPU trace is integrated with Equation 4.11 and

$$E_{\text{GPU+host}} = E_{\text{GPU}} + E_{\text{CPU,host}}. \quad (4.10)$$

For one session at the (512, 64) operating point, the board trace E_{GPU} is also reported on its own, to compare the two accelerators device to device.

Tenstorrent. A Python profiler launches the C++ benchmark, polls device power through `tt-smi`, and records the host’s cumulative RAPL counter. The external `tt-smi` call dominates the polling cadence: for the inspected (512, 32) run, the median gap between samples is about 0.73 s. `tt-smi` reports the chip power telemetry of the Wormhole firmware [29], whereas `nvidia-smi` reports the power of the entire A100 board [19], so the two device boundaries are not identical.

Integration. For a sampled power trace with samples (t_i, P_i) , $i = 1, \dots, m$, energy is estimated with the trapezoidal rule on the recorded timestamps,

$$E_{\text{trace}} = \sum_{i=2}^m \frac{P_{i-1} + P_i}{2} (t_i - t_{i-1}), \quad (4.11)$$

while cumulative RAPL counters use the difference in Equation 4.9. Energy is never reconstructed from an unweighted mean of power samples. Average power, where reported, is derived as integrated energy divided by elapsed session time. Amortized energy per query is

$$E_{\text{query}} = \frac{E_{\text{run}}}{100,000}, \quad (4.12)$$

where E_{run} is the median session energy; it includes the initialization share of the session and is not the energy of an isolated query.

4.8 Chapter Summary

Throughput is the median of ten 10,000-query search repetitions, recall is evaluated outside all timed and monitored intervals, and energy is the median over ten complete 100,000-query sessions that include initialization. Because the Tenstorrent batch-level union changes the candidate set, cross-platform comparisons are matched by measured Recall@10. The next chapter reports the resulting throughput, stage decomposition, and energy.

Chapter 5

Experimental Results

This chapter compares throughput at measured recall, analyzes the effect of N_{list} and N_{probe} , decomposes the Tenstorrent search path, and reports workload energy. Throughput is the median of ten search-only repetitions, whereas energy covers the complete 100,000-query session of Equation 4.6, including initialization.

5.1 CPU, GPU, and Tenstorrent Comparison

Figure 5.1 shows QPS against measured Recall@10 on a logarithmic throughput axis. Placing recall rather than N_{probe} on the horizontal axis allows the Tenstorrent batch-union implementation to be compared with per-query IVF search without assuming equivalent candidate sets. Table 5.1 extracts the four recall-matched operating points of Table 4.2.

Table 5.1. Median search throughput at the recall-matched operating points of Table 4.2. The ratio is baseline QPS divided by Tenstorrent QPS, so values above one favor the baseline.

Target	Median QPS				Ratio to TT		
	CPU (56)	CPU (112)	A100	TT	CPU (56)	CPU (112)	A100
60%	151,858	266,184	1,583,712	49,720	3.05	5.35	31.85
78%	43,864	80,075	544,703	21,907	2.00	3.66	24.86
88%	11,466	22,079	160,799	10,762	1.07	2.05	14.94
95%	3,190	5,822	43,271	5,550	0.57	1.05	7.80

The GPU is the fastest configuration across the whole recall range, by a factor of 31.9 over Tenstorrent at the 60% target and 7.8 at the 95% target. Tenstorrent is also slower than both CPU configurations up to about 88% recall: the 56-core CPU is 3.05 times faster at 60% and reaches parity at 88%, while the 112-core CPU remains 2.05 times faster at 88%. The crossover with the 56-core CPU lies near 90% recall: Tenstorrent at (512, 16) reaches 92.4% recall at 7,229 QPS, whereas the CPU reaches 91.4% at 6,090 QPS with (512, 32). At the 95% target, Tenstorrent is 1.74 times faster than the 56-core CPU and within 5% of the 112-core CPU. These comparisons hold against the evaluated grid: near 95% recall the only Faiss

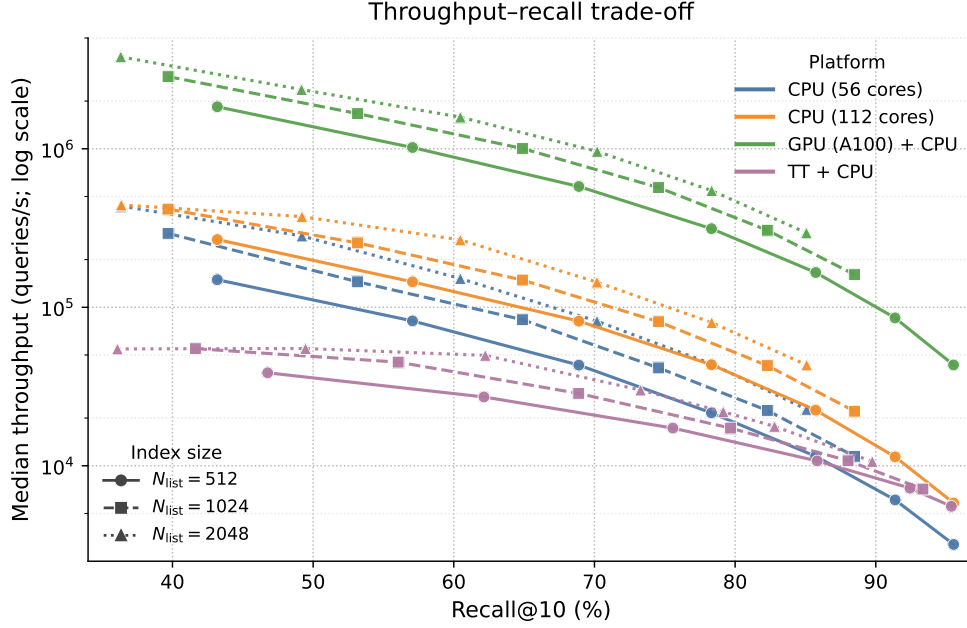


Figure 5.1. Queries per second versus measured Recall@10. Each point is the median of ten repetitions, and lines connect N_{probe} values within one N_{list} sweep. The Tenstorrent (2048, 12) energy-matching point is included; other exploratory $N_{\text{probe}} = 12$ runs are not.

point is (512, 64), whereas Section 5.2 shows that Faiss is faster with more lists, so configurations with more lists and larger N_{probe} , which were not evaluated, could narrow or reverse the CPU comparison. Within the evaluated grid, Tenstorrent’s relative position improves steadily with recall, because its throughput falls more slowly than that of the baselines as search depth grows. When N_{probe} doubles, Faiss scans twice as many lists for every query, and its throughput roughly halves: at 512 lists, going from $N_{\text{probe}} = 16$ to 32 lowers it by a factor of 1.89 on the 56-core CPU. On Tenstorrent, the work per batch is set by the union of the lists selected by its queries, and the additional lists increasingly overlap with those already selected by other queries, so the union grows by much less than a factor of two (Section 5.2). At 512 lists, its pages grow by a factor of 1.51 from $N_{\text{probe}} = 8$ to 16 and of 1.31 from 16 to 32, and the throughput falls by almost the same factors, 1.49 and 1.30. At low recall, however, Tenstorrent is limited by a fixed per-batch cost, also analyzed in Section 5.2.

The 112-core CPU is faster than the 56-core CPU at every configuration, but the speed-up depends on the amount of work per query. It is small when little candidate work is available, only 1.02 at (2048, 1), 1.33 at (2048, 2), and 1.42 at (1024, 1), because per-call overheads then dominate. With more work it lies between 1.75 and 2.01 without a monotonic trend, peaking at (512, 8). Near-linear and occasionally slightly super-linear scaling is consistent with the larger aggregate cache and memory bandwidth available to the full node.

Run-to-run variation is small compared with the differences between platforms. Across the complete grid, the QPS RSD of Equation 4.5 is at most 4.64% for the

56-core CPU, 5.89% for the 112-core CPU, 2.43% for the GPU, and 1.46% for Tenstorrent, and the ten repetitions show no systematic first-repetition slowdown. The medians in Figure 5.1 and Table 5.1 are therefore not driven by individual repetitions.

5.2 Impact of N_{list} and N_{probe}

Within every sweep, increasing N_{probe} raises recall and lowers throughput. The choice of N_{list} , however, affects the platforms differently.

Faiss favors finer partitions. At matched recall, the Faiss baselines are faster with more lists. Near 78% recall, the GPU reaches 544,703 QPS at (2048, 16) but only 313,161 QPS at (512, 8), a factor of 1.74; the gain is 2.03 on the 56-core CPU and 1.84 on the 112-core CPU. Smaller lists reduce the candidate volume per probe more than the larger coarse scan costs.

The batch union raises Tenstorrent recall. Table 5.2 compares Tenstorrent and Faiss recall at the same configuration. Because every query is also scored against lists selected by other queries in its batch, Tenstorrent is more accurate at almost every point, by up to 7.52 percentage points at (512, 8). The gain first grows with N_{probe} and then shrinks as recall approaches saturation. It also decreases with N_{list} : the maximum is 7.52 points for 512 lists, 5.74 for 1024, and 4.67 for 2048. This is consistent with lists contributed by other queries being less likely to contain a query’s true neighbors when the partition is finer.

Table 5.2. Recall@10 difference, Tenstorrent minus Faiss CPU, in percentage points at equal $(N_{\text{list}}, N_{\text{probe}})$.

N_{list}	N_{probe}					
	1	2	4	8	16	32
512	+3.58	+5.08	+6.68	+7.52	+6.71	+4.00
1024	+1.95	+2.89	+4.00	+5.12	+5.74	+4.85
2048	−0.27	+0.25	+1.78	+3.10	+4.46	+4.67

The union multiplies the scanned data. The recall gain has a cost. Figure 5.2 and Table 5.3 compare the fraction of the database against which each query is scored on Tenstorrent, whose queries scan the whole batch union, with the fraction that Faiss scans through the query’s own N_{probe} lists. Both use the lists selected by the Tenstorrent coarse search, logged on the host for one repetition of 10,000 queries; the batching is deterministic, and a repeated run produced identical unions. At equal parameters, a Tenstorrent query is scored against 12 to 32 times as many vectors as a Faiss query. At the four energy operating points it scans 6.7%, 18.3%, 41.0%, and 81.7% of the database, so at the 95% target the fine search is close to an exhaustive scan.

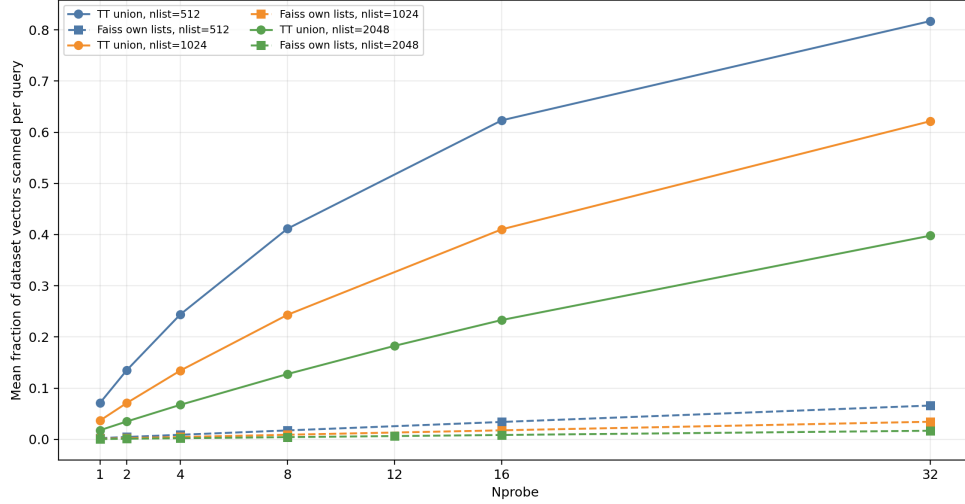


Figure 5.2. Mean fraction of the database scanned per query as a function of N_{probe} : the batch union on Tenstorrent (solid) and the query’s own lists as in Faiss (dashed), for each N_{list} .

Table 5.3. Mean fraction of the database scanned per query on Tenstorrent, and the factor by which it exceeds the vectors in the query’s own N_{probe} lists. A factor of 32 means that the 32 queries of a batch share no lists.

N_{list}		N_{probe}						
		1	2	4	8	12	16	32
512	scanned	7.1%	13.5%	24.4%	41.1%	–	62.3%	81.7%
	factor	30.6	29.5	27.3	23.6	–	18.3	12.4
1024	scanned	3.7%	7.1%	13.4%	24.3%	–	41.0%	62.1%
	factor	31.2	30.6	29.4	27.1	–	23.3	18.1
2048	scanned	1.8%	3.5%	6.7%	12.8%	18.3%	23.3%	39.8%
	factor	31.6	31.3	30.7	29.6	28.5	27.4	23.7

The factors close to 32 show that consecutive queries in dataset order share almost no lists. At (2048, 4), for example, the 128 lists requested by a batch contain a median of 124 distinct lists. Overlap appears only when the union covers a large part of all lists, as at (512, 32), where 1,024 selections contain a median of 412 distinct lists, or 80% of the 512. This explains the recall gain of Table 5.2, which is bought with a much larger scan, and why the gain falls with finer partitions, where each extra list covers a smaller part of the database. It also explains why Tenstorrent’s position against the CPUs improves with recall: its scanned volume approaches the whole database and grows slowly, whereas the per-query scan of Faiss keeps growing with N_{probe} .

The union is nevertheless the right choice for the device. Every query must still be scored against its own N_{probe} lists, and each fine-search step multiplies a full 32-query tile regardless of how many rows are useful. Processing the queries separately would therefore fetch and multiply every selected list once per query that requests it, whereas the union fetches each distinct list only once per batch. The number of pages fetched falls by the overlap factor, the ratio of requested to distinct lists, and the extra candidates that each query receives come from lists fetched for other queries at no additional matrix multiplication. In dataset order, however, the overlap factor is only 1.0–2.5, so the saving is small; grouping queries that select similar lists into the same batch would raise it.

Tenstorrent sweeps converge at high recall. Above about 85% recall, the three Tenstorrent sweeps nearly coincide: (512, 8), (1024, 16), and (2048, 32) all run at about 10,700 QPS and reach 85.83%, 88.03%, and 89.75% recall respectively. In this regime N_{list} mainly shifts recall at similar cost, and 2048 lists give the best trade-off.

Tenstorrent has a fixed per-batch floor. At low N_{probe} , Tenstorrent throughput stops increasing: (1024, 1), (2048, 1), and (2048, 2) all run at about 54,800 QPS, or about 182 ms per 10,000 queries, even though their candidate volumes differ. This corresponds to roughly 0.58 ms per 32-query batch that does not depend on N_{probe} . At 2048 lists the most likely cause is the final aggregation. The aggregation core processes the batches one after another, and for every batch it merges one top-32 result from each of the 63 workers, including workers without assigned lists (Section 3.5.2). In the batch trace of Section 5.3.1 this takes about 0.44 ms per batch, close to the floor. At larger N_{probe} the workers need several milliseconds per batch, so the aggregation overlaps with their work and is not a bottleneck; at low N_{probe} the workers finish a batch sooner than the aggregation, which then becomes a likely bottleneck. This floor caps Tenstorrent’s low-recall throughput and explains much of its gap to the CPUs at the 60% target. Load imbalance contributes as well: at $N_{\text{probe}} = 1$ a batch union holds at most 32 lists, so at most 32 of the 63 workers can receive work, and in the logged schedule, which does not split lists, the busiest worker holds a median of 3.7–4.2 times the mean load; at 1024 lists this imbalance alone may account for most of the floor. The stage decomposition below covers only $N_{\text{list}} = 512$, where candidate work dominates, so it does not isolate this cost.

5.3 Tenstorrent Stage Decomposition

Figure 5.3 shows the six stages of Equation 4.7 for $N_{\text{list}} = 512$ and $N_{\text{probe}} \in \{1, 8, 16, 32\}$. Each segment is the median of that stage over ten repetitions of 10,000 queries, after one warm-up invocation, so a stack height is a sum of stage-wise medians. Figure 5.4 omits TT fine search and rescales the vertical axis to expose the remaining stages.

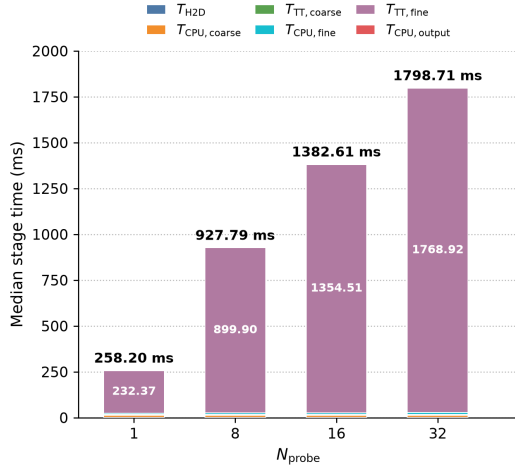


Figure 5.3. Median Tenstorrent stage times for $N_{\text{list}} = 512$, including TT fine search.

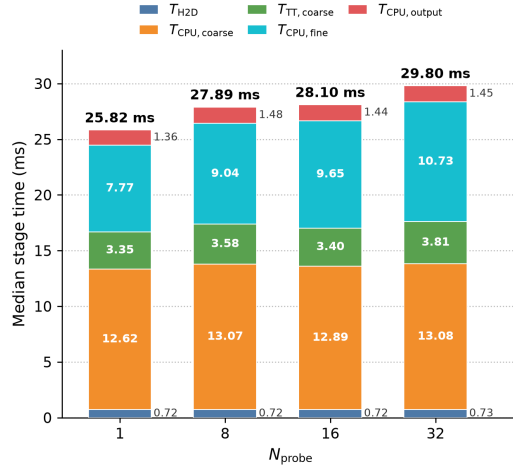


Figure 5.4. Rescaled view of the same stage times with TT fine search omitted.

TT fine search dominates. As N_{probe} increases from 1 to 32, the total rises from 258.20 to 1,798.71 ms and fine search from 232.37 to 1,768.92 ms, accounting for 90.0%, 97.0%, 98.0%, and 98.3% of the four totals. The totals agree with the independent throughput measurements: 1,798.71 ms per 10,000 queries corresponds to about 5,560 QPS, against a median of 5,550 QPS. All other stages together grow only from 25.82 to 29.80 ms: CPU coarse configuration stays between 12.62 and 13.08 ms, TT coarse search between 3.35 and 3.81 ms, and CPU fine preparation grows from 7.77 to 10.73 ms. Optimizing these stages would therefore have little effect at this N_{list} . Host timing cannot tell whether compute, DRAM traffic, NoC communication, circular-buffer backpressure, or synchronization limits the fine stage, which motivates the device trace below.

5.3.1 Device-Kernel Timeline

Figure 5.5 shows a device-profiler capture with $N_{\text{list}} = 512$ and $N_{\text{probe}} = 8$, which reaches 85.8% recall, zoomed into the last three measured batches, labeled B1 to B3. It covers the aggregation core and three consecutive fine-search workers. The workers were taken in coordinate order, not selected by behavior, so the figure is illustrative rather than a statistical sample of all workers. The figure separates each core into its reader (NCRISC) and its three compute threads (TRISC unpack, math, and pack). Each worker scope is labeled with its batch and list task, so a bar covers the processing of one task: a complete list or a chunk of at most 128 pages.

Four observations follow from the zoomed trace.

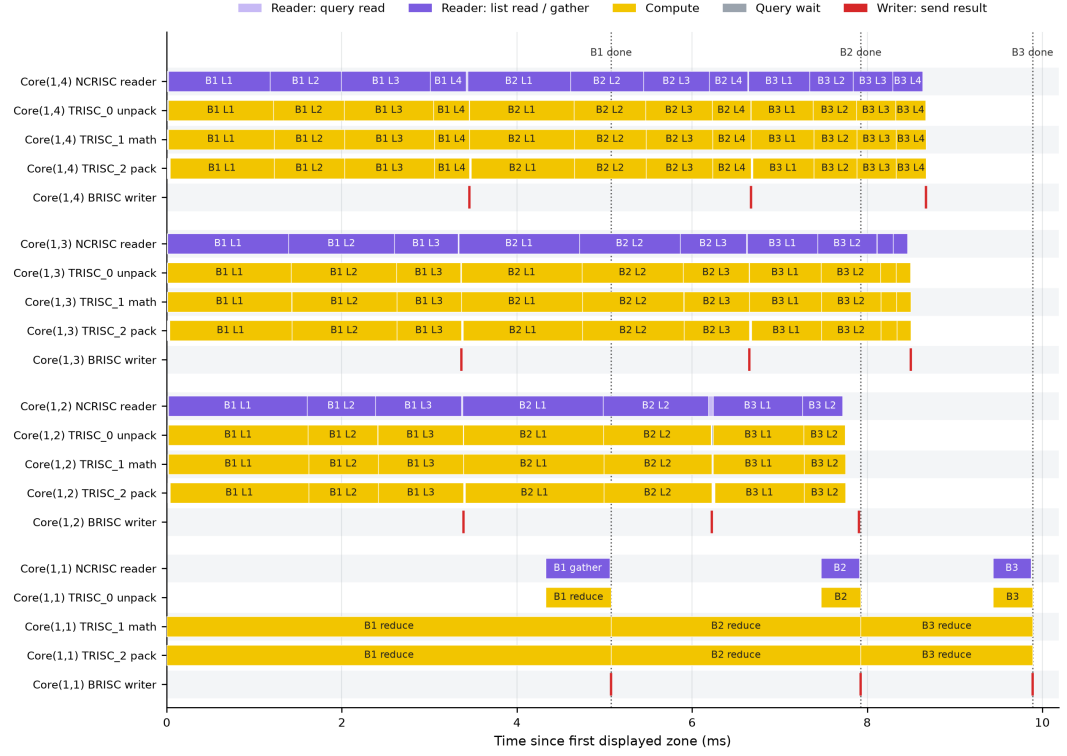


Figure 5.5. Zoomed fine-search pipeline at $(N_{\text{list}}, N_{\text{probe}}) = (512, 8)$ for the last three 32-query batches (B1 to B3), with one row per RISC processor of the aggregation core (profiler core (1, 1)) and of three workers (profiler cores (1, 2) to (1, 4)). A worker zone labeled $Bb Li$ covers the i -th list task of batch b ; red ticks mark the writer sending a result, and dotted lines mark the completion of each batch on the aggregation core.

1. **Workers do not stop between batches.** Each worker starts the first list of the next batch within a few tens of microseconds of finishing the previous one. The aggregation core gathers B1 between about 4.3 and 5.1 ms, when the three workers shown are already processing B2. Batch reduction is therefore off the workers' critical path and introduces no per-batch barrier.
2. **Reader and compute advance in lock-step.** On every worker, each reader scope ends a few tens of microseconds before the corresponding compute scope, and the unpack, math, and pack scopes coincide. The pipeline overlaps the reading and scoring of the same list, and its fill latency at the start of a batch is only about 30–40 μ s. Because each scope spans a whole task, the trace still cannot tell whether compute waits for data or the reader is throttled by full circular buffers.
3. **Worker imbalance sets batch completion.** The three workers finish B1 within about 0.1 ms of each other, but drift apart later, completing B3 at about 7.8, 8.5, and 8.7 ms. The aggregator's B1 gather starts about 0.9 ms after the last of the three workers shown has finished B1, which is consistent with other workers finishing later. As batch completion depends on the slowest of the 63 workers, the greedy assignment of coarse list tasks translates directly into idle capacity.
4. **Aggregation is short and overlapped.** The aggregator's gather and unpack scopes last about 0.73, 0.44, and 0.44 ms for the three batches. Its math and pack scopes span almost the entire interval because they include waiting for worker results, not because reduction is expensive. At this configuration the aggregation is therefore not an issue, but at low N_{probe} , where the workers finish a batch faster, the same per-batch cost becomes a likely bottleneck (Section 5.2).

The 2.85 ms between the completions of B1 and B2 matches the 2.96 ms per batch measured by the stage timing at this configuration (927.79 ms over 313 batches), so the zoomed batches are representative. Batch B3 is visibly shorter on all workers because it is the final batch of the repetition, which holds only 16 valid queries (Equation 4.4) and therefore a smaller list union. Each worker's writer sends its fixed-size local result in under 1 μ s per batch, shown as red ticks.

Overall, the zoomed trace shows that fine-search time is spent inside the per-list reader–compute loop of each worker and is extended by imbalance across workers, whereas batch transitions and aggregation contribute little.

Single-list zoom. Figure 5.6 zooms further, into the processing of 16 consecutive pages (P0 to P15) of one inverted list on worker (1,2), captured with $N_{\text{list}} = 512$ and $N_{\text{probe}} = 1$. Reader zones cover the DRAM fetch of each page, and compute zones cover the unpacking, scoring, and top- k update of each page.

1. **Compute costs about 7 μ s per page.** When data is available (P1 to P4 and P8 to P14), each page occupies the math and pack threads for about 7 μ s, while the reader fetches the next page mostly in under 3 μ s shortly before it is

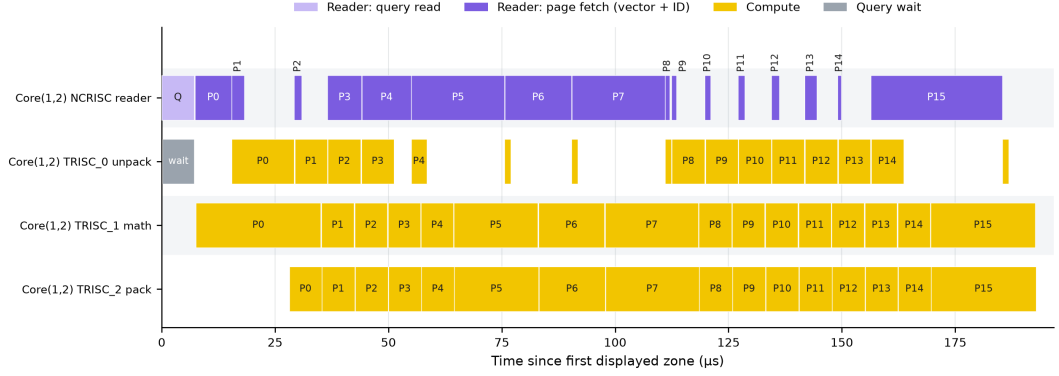


Figure 5.6. Processing of 16 consecutive pages (P0 to P15) of one inverted list on worker (1, 2) at $(N_{\text{list}}, N_{\text{probe}}) = (512, 1)$. The reader row shows the query fetch and the DRAM fetch of each page; the compute rows show the work on each page and the initial wait for the query.

needed. In this regime the pipeline is limited by the per-page compute step, not by arithmetic (see below), and the reader waits for free buffer space. A page requires only four 32×32 tile multiplications, so the $7 \mu\text{s}$ suggests that most of the compute time is spent outside the matrix multiplication, in the top- k update.

2. **Slow fetches create bubbles.** The fetches of P5, P6, P7, and P15 take 15–29 μs instead of under 3 μs . The unpacker then has no input, and the math and pack zones of these pages stretch to 15–23 μs . Start-up adds a further bubble: the query fetch and the first page read delay the pack thread until about 28 μs , and the first page is complete only at about 35 μs .
3. **About 40% of the time is waiting.** Sixteen pages at about 7 μs account for roughly 115 μs of the 193 μs shown, so compute waits for data for about 40% of the time.
4. **The buffer hides little variation.** The reader runs only about one page ahead of compute, as expected from the double-buffered candidate buffers (Section 3.5.1), so a single slow fetch stalls the pipeline immediately, whereas the 8–11 μs fetches of P3 and P4 are hidden because they start early enough. Deeper buffering does not solve this, however: in single exploratory runs at (512, 32) with an instrumented build, letting the reader prefetch eight pages instead of two made fine search about 7% slower (2,006 against 1,880 ms per 10,000 queries), consistent with the additional outstanding reads adding to the contention between workers for the DRAM channels.

Arithmetic intensity. The “compute” in the trace is not arithmetic. Each page fetches 12,288 bytes, 8,192 for the 32 vectors of 128 FP16 dimensions and 4,096 for the identifier tile, and performs one 32×128 by 128×32 multiplication of 262,144 floating-point operations. Even with all 32 query rows in use, this is only 21 operations per byte, an order of magnitude below the balance point of

the Wormhole n150, about 257 operations per byte for its 74 FP16 TFLOP/s and 288 GB/s [30]. Most rows are not even useful: only the queries that selected the list need its scores, which on average is one row in 12 to 31 (Table 5.3), so the useful intensity is only 0.7–1.7 operations per byte. The fine search is therefore memory-bound in the roofline sense, and the matrix engine is never its limit: at (512, 32) it runs at about 1.4 TFLOP/s, 2% of the peak. The DRAM is not saturated either, since the 9.5 million pages fetched in 1.77 s amount to about 66 GB/s, 23% of the peak bandwidth. The time per page is set instead by the work around the multiplication, chiefly the top- k update of the scores and identifiers, and by the latency of the fetches, which become slow and irregular when many workers read at the same time. Both depend on the number of pages processed, which is why reducing that number matters more than faster arithmetic.

Growth with N_{probe} . The window was captured at $N_{\text{probe}} = 1$, which is the most favorable case for data availability: the union of a 32-query batch then contains at most 32 lists, so at most 32 of the 63 workers are active. Three effects are expected to enlarge the problem as N_{probe} grows.

1. **More concurrent readers.** From $N_{\text{probe}} = 2$ onwards, the union can exceed the 63 workers, so all of them fetch pages at the same time and compete for the same DRAM channels and NoC links. The trace does not show why the fetches of P5 to P7 and P15 are slow, but if contention is the cause, such fetches become more frequent and longer.
2. **More pages per worker.** Each worker processes more list tasks per batch, so every stall is repeated over more pages. Even at an unchanged fraction of waiting per page, the absolute waiting time grows with the number of pages scanned.
3. **Constant compute cost per page.** The approximately 7 μs per page does not decrease with N_{probe} , so the phases limited by the per-page compute step also grow in proportion to the pages scanned.

Consistently with these effects, TT fine search grows from 232 to 1,769 ms per 10,000 queries between $N_{\text{probe}} = 1$ and 32 (Section 5.3). Reducing the data fetched per query and the cost of the per-page top- k update are the two main directions for improving fine search.

5.4 Workload Energy

Table 5.4 and Figure 5.7 report the median energy of ten complete 100,000-query sessions at each recall-matched point, amortized per query with Equation 4.12, together with the interquartile range (IQR) and the relative standard deviation (RSD) across the ten sessions. For Tenstorrent, the energy is that of the complete system, device and host together.

The GPU system uses the least energy at every target, 9.3–20.1 mJ per query. Tenstorrent is second, with 30.5–46.9 mJ per query, which is 2.3 times the GPU value at the 95% target and 3.2–3.8 times at the lower targets. Both CPU configurations

Table 5.4. Workload energy for 100,000-query sessions, including initialization. Energy is the median of ten sessions, with the IQR and RSD across them, and its amortized per-query value. Configuration denotes $(N_{\text{list}}, N_{\text{probe}})$.

Architecture	Config.	Recall@10	Energy (J)	IQR (J)	RSD	mJ/query
CPU (56 cores)	(2048, 4)	60.46%	6,995.69	6,927–7,090	2.36%	69.957
	(2048, 16)	78.33%	7,981.57	7,901–8,070	1.93%	79.816
	(1024, 32)	88.49%	9,211.55	9,140–9,351	1.40%	92.116
	(512, 64)	95.54%	21,872.36	21,831–21,988	0.67%	218.724
CPU (112 cores)	(2048, 4)	60.46%	6,915.42	6,835–7,002	2.65%	69.154
	(2048, 16)	78.33%	7,526.78	7,467–7,747	2.67%	75.268
	(1024, 32)	88.49%	7,292.36	7,270–7,371	1.50%	72.924
	(512, 64)	95.54%	14,928.76	14,741–15,060	1.46%	149.288
GPU (A100) + CPU	(2048, 4)	60.47%	932.97	776–1,085	36.60%	9.330
	(2048, 16)	78.33%	940.54	847–1,019	24.01%	9.405
	(1024, 32)	88.49%	1,064.14	1,021–1,113	5.72%	10.641
	(512, 64)	95.53%	2,010.66	1,989–2,131	6.23%	20.107
TT + CPU	(2048, 4)	62.24%	3,050.09	3,036–3,076	1.78%	30.501
	(2048, 12)	79.16%	3,542.92	3,525–3,562	3.85%	35.429
	(1024, 16)	88.03%	3,438.38	3,420–3,457	1.32%	34.384
	(512, 32)	95.37%	4,692.64	4,669–4,741	1.93%	46.926

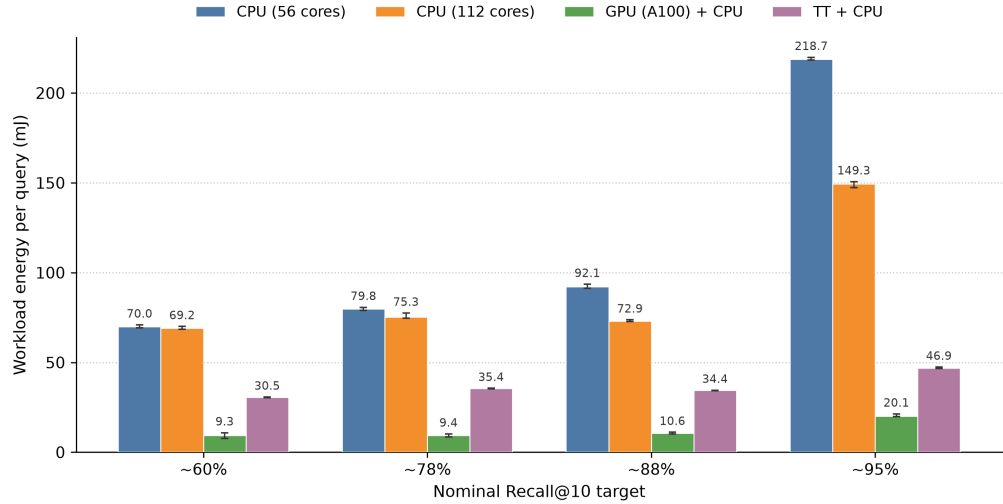


Figure 5.7. Amortized workload energy per query at approximately matched Recall@10, including initialization. Bars show the median of ten sessions and whiskers the interquartile range.

use the most: 69–92 mJ per query up to the 88% target and 149–219 mJ at 95%. Tenstorrent therefore uses 53–63% less energy than the CPUs up to the 88% target, and 78.5% and 68.6% less than the 56-core and 112-core CPUs at 95%, where the GPU uses 90.8% and 86.5% less.

The CPU and Tenstorrent values are reproducible across sessions, with an RSD of at most 2.67% on the CPUs and 3.85% on Tenstorrent. The GPU values vary more at low recall, with an RSD of 36.6% at 60% and 24.0% at 78%, because these sessions last less than 3 s and are almost entirely initialization; at the 88% and 95% targets the RSD is about 6%.

The host dominates Tenstorrent’s energy. The Wormhole device itself uses 327–850 J per session, 23.9–33.6 W on average, or only 10.7–18.1% of the system energy. The remaining 152–200 W is drawn by the dual-socket EPYC 7301 host, whose combined rated TDP of about 310–340 W is less than half that of the 700 W CPU node. The energy of the Tenstorrent system is therefore set mainly by its host.

For a device-to-device view at the 95% target, the A100 board trace of one session is examined on its own; the board power is integrated in every GPU session, but only this trace is analyzed separately here. The board draws about 66 W when idle and about 377 W on average while searching, with a peak of 386 W, below its 550 W limit; the high-power interval lasts 2.2 s, matching the 2.31 s search time. Over the session the board uses 1,066 J, or 10.7 mJ per query, against 8.5 mJ for the Wormhole chip. The two devices therefore use comparable energy for this workload, keeping in mind that the chip telemetry of the Wormhole excludes the board-level losses that the A100 measurement includes: the A100 draws far more power, but finishes the search about 7.8 times faster.

The GPU system is not dominated by its host to the same extent. At the 95% target, the 1,066 J of the A100 board are about half of the 2,011 J median energy of the complete A100 system, whereas the Wormhole chip accounts for only 18% of the Tenstorrent system. Since the board figure comes from a single session and the system figure is a median over ten, this split is approximate, but it shows that the energy gap between the two systems lies mainly in their host energy rather than in the accelerators. The Tenstorrent host is not more power-hungry, drawing 152 W at this target against about 218 W for the A100 host, but it is powered for a session 5.8 times longer. The Wormhole device thus reaches a device energy comparable to that of the A100 while being a lower-cost accelerator, and shortening the session, through faster initialization and search, is the most direct way to reduce the energy of the Tenstorrent system.

Initialization dominates at low recall. Table 5.5 relates each session to its search time, estimated as $Q_{\text{run}}/QPS_{\text{reported}}$. At the 60% target, search occupies only 2–4% of every CPU and GPU session and 15% of the Tenstorrent session; the remainder is process start-up, dataset loading, list assignment, and, on the accelerators, device setup and index transfer. The per-query values in Table 5.4 therefore measure the cost of a complete 100,000-query job from a cold start, not the energy of search alone.

The time spent outside search differs strongly between the platforms. On the

GPU system it is 2.0–2.8 s for every configuration. On the CPUs it grows with N_{list} , from 3.0–7.2 s at 512 lists to 9.3–10.8 s at 1024 and 17.4–17.9 s at 2048, and at 1024 and 2048 lists it is almost the same on 56 and 112 cores. On Tenstorrent, whose host also performs the assignment, it grows from 7.3 s at 512 lists to 8.2 s at 1024 and 11.6–11.7 s at 2048. This is consistent with the assignment of the 1.18 million database vectors to their nearest centroids, whose cost is proportional to N_{list} , running on the host CPU without benefiting from additional cores, whereas the GPU performs it on the device. The low-recall CPU and Tenstorrent energy is therefore dominated by initialization rather than by search.

Table 5.5. Composition of the energy sessions. Search time is $100,000/\text{QPS}_{\text{reported}}$; session time is the median duration of the complete session, which includes the search; and average power is the median workload energy divided by the median session time.

Architecture	Target	Search (s)	Session (s)	Search share	Avg. power (W)
CPU (56 cores)	60%	0.66	18.171	3.6%	385.0
	78%	2.28	19.667	11.6%	405.8
	88%	8.72	17.983	48.5%	512.2
	95%	31.35	34.376	91.2%	636.3
CPU (112 cores)	60%	0.38	18.211	2.1%	379.7
	78%	1.25	19.126	6.5%	393.5
	88%	4.53	15.314	29.6%	476.2
	95%	17.18	24.395	70.4%	612.0
GPU (A100) + CPU	60%	0.06	2.828	2.2%	329.9
	78%	0.18	2.795	6.6%	336.5
	88%	0.62	2.862	21.7%	371.8
	95%	2.31	4.335	53.3%	463.8
TT + CPU	60%	2.01	13.626	14.8%	223.8
	78%	4.56	16.285	28.0%	217.6
	88%	9.29	17.486	53.1%	196.6
	95%	18.02	25.294	71.2%	185.5

More CPU cores do not cost more energy. The 56-core CPU never uses less energy than the 112-core CPU. At the 60% target the two are within 1.2%, because both sessions are almost entirely initialization, which does not benefit from more cores, so the node draws about 380 W in either configuration. As search takes a larger share of the session, the 56-core CPU uses 6% more energy at 78%, 26% more at 88%, and 47% more at 95%. Doubling the cores does not double the power, because most of the node’s power is fixed: both sockets, with their caches, memory controllers, and idle cores, are powered for the whole session regardless of how many cores search. Moreover, with 56 cores the active cores can likely run at higher clock frequencies and draw more power each, while the idle ones still draw some; with 112 cores all of them are active, but each draws less, so the average power changes little. Assuming about 380 W during initialization, the average power during the search at the 95% target is roughly 660 W with 56 cores and 710 W with 112, only about 7%

more, while the search takes 31.35 s instead of 17.18 s. The 112-core search therefore spends about 12 kJ instead of 21 kJ: it pays the fixed power of the node for a much shorter time.

5.4.1 Workload Throughput and Energy

To view speed and energy on the same workload boundary, Figure 5.8 uses the workload throughput

$$\Theta_{\text{workload}} = \frac{Q_{\text{run}}}{t_{\text{session}}} = \frac{100,000}{t_{\text{session}}}, \quad (5.1)$$

which, unlike the search-only QPS of Section 5.1, includes initialization.

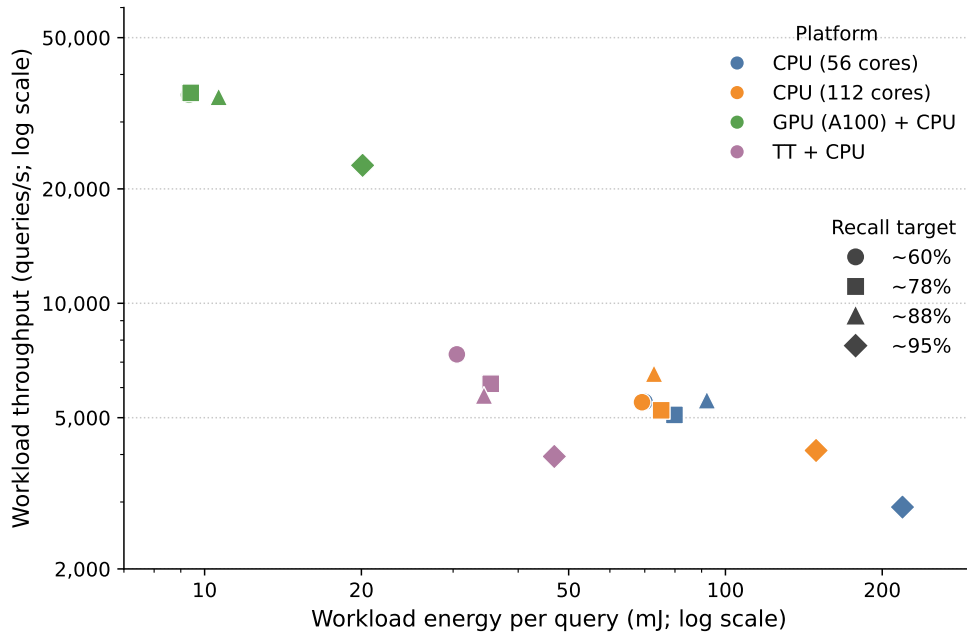


Figure 5.8. Workload throughput, including initialization, versus amortized workload energy per query at approximately matched Recall@10 targets.

The GPU system lies in the upper left at every target: it completes the workload fastest, at 35,000–36,000 queries per second up to the 88% target, and uses the least energy. The Tenstorrent and CPU sessions reach similar workload throughput, between about 2,900 and 7,300 queries per second, because initialization takes a large part of each of them, but Tenstorrent uses 53–79% less energy. At the 95% target, the GPU, 112-core, Tenstorrent, and 56-core sessions correspond to 23,068, 4,099, 3,954, and 2,909 queries per second. These ratios are smaller than the search-only differences in Table 5.1 because initialization occupies part of every session.

5.5 Summary of Results and Limitations

The main observations are:

- The GPU has the highest search throughput at every recall level, from 31.9 times Tenstorrent at 60% to 7.8 times at 95%.
- Tenstorrent is slower than both CPU configurations up to about 88% recall; at 95% it is 1.74 times faster than the 56-core CPU and on par with the 112-core CPU against the evaluated Faiss grid.
- The batch-level union raises Tenstorrent recall by up to 7.5 percentage points at equal parameters, but each query is scored against 12–32 times as many vectors as in Faiss, and against 81.7% of the database at the 95% target. Consecutive queries share almost no lists. At low N_{probe} , Tenstorrent throughput is capped by a per-batch cost of about 0.58 ms.
- Fine search accounts for 90–98% of the Tenstorrent search path at $N_{\text{list}} = 512$. Inside it, reader and compute advance in lock-step on each list, batch transitions and aggregation are overlapped with worker execution, and batch completion is set by the slowest worker. Within a list, compute costs about $7 \mu\text{s}$ per page and waits for slow DRAM fetches for about 40% of the time.
- The GPU system uses the least workload energy at every recall target, Tenstorrent 2.3–3.8 times as much, and the CPUs the most. Tenstorrent uses 53–79% less energy than the CPUs, and its energy is dominated by its host, the device accounting for only 11–18% of it. On the CPU node, 112 cores never use more energy than 56, because the node’s power is drawn for the whole session and more cores finish sooner.

These results are subject to the following limitations:

- **Workload boundary.** Energy includes initialization, so the per-query values depend on workload length and do not measure search energy alone.
- **Quality matching.** Tenstorrent and Faiss process different candidate sets, so cross-platform points are matched only approximately by recall, and the effect of alternative query groupings is not evaluated.
- **Platforms.** The three systems use different hosts and hardware generations, including a 2017 AMD EPYC 7301 host for Tenstorrent, and their device power sensors cover different parts of the accelerator (chip telemetry on Tenstorrent, the whole board on the A100). The comparison is between complete systems, not between processor architectures in isolation.
- **Dataset scale.** The 16-bit database occupies about 237 MB, which is small for a vector-search deployment and close to the combined last-level cache of the dual-socket CPU node. The dataset was chosen as a standard benchmark with published exact neighbors, which the recall-matched comparison requires. Collections of the size that motivates this work, such as large RAG corpora, would not fit on one device: in the current layout the 12 GB of the Wormhole n150 hold about 30 million vectors (Section 6.2); larger collections would require distributing the index over several devices or streaming lists from host memory, which are natural extensions of this work.

- **Bottleneck attribution.** The single-list trace covers 16 pages of one list on one worker at $N_{\text{probe}} = 1$. It separates fetch from compute but not the matrix multiplication from the top- k update, and it does not explain why some DRAM fetches are slow.
- **Quantization.** The Tenstorrent implementation uses full FP16 vectors with FP32 accumulation; no conclusion is drawn about 8-bit or other quantized fine-search paths.

Chapter 6

Conclusions and Future Work

This thesis presented, to the best of our knowledge, the first approximate nearest-neighbor search implementation for Tenstorrent hardware: an IVF-Flat pipeline on a Wormhole n150 that combines device-side coarse search, host-side scheduling of the lists selected by each 32-query batch, and distributed fine search with hierarchical top- k selection. It was evaluated on glove-100-angular against Faiss on two CPU configurations and an NVIDIA A100 system, at matched Recall@10, in terms of throughput, workload energy, and the cost of each stage of the search path. This chapter answers the research questions, summarizes the limitations, and outlines future work.

6.1 Answers to the Research Questions

RQ1: Which ANN index is most suitable for Tenstorrent? IVF-Flat. Its fine search reduces to dense multiplications between query tiles and contiguous blocks of database vectors, which map directly onto Tensix tile operations and explicit DRAM transfers, whereas graph-based indexes depend on pointer chasing and query-dependent control flow (Chapter 2). This is an argued choice rather than a measured comparison, since no other index was implemented. The implementation showed that the regular part of the search maps well onto the device, but also that the irregular parts of IVF, namely list selection, list sizes, and scheduling, remain the difficult part of the mapping: they were placed on the host and they surface as worker imbalance on the device.

RQ2: Can IVF search be implemented efficiently on Tenstorrent? It can be implemented, but not yet efficiently. The pipeline reaches 95.4% Recall@10 and runs at 5,550 queries per second at that recall, and grouping queries into 32-query batches lets the fine search use full tiles. The same batching, however, makes every query in a batch scan the union of the lists selected by the whole batch. Because consecutive queries share almost no lists, each query is scored against 12 to 32 times as many vectors as its own lists contain, and against 81.7% of the database at the 95% operating point. This raises recall by up to 7.5 percentage points at equal parameters, but at high recall it turns IVF search into a nearly exhaustive scan. The union itself saves data movement, since each distinct list is fetched once per

batch instead of once per query that requests it, but with queries in dataset order this saving is small. Inside the fine search, one page of candidates costs about $7\ \mu\text{s}$ of compute, which is far more than its four tile multiplications, and in the profiled window compute waits for DRAM fetches for about 40% of the time because the reader runs only about one page ahead.

RQ3: What recall and throughput does Tenstorrent achieve? The A100 is the fastest system at every recall level, by 31.9 times at 60% recall and 7.8 times at 95%. Tenstorrent is slower than both CPU configurations up to about 88% recall; at the 60% target this is largely due to a fixed cost of about 0.58 ms per batch that caps its throughput at low N_{probe} . Its throughput, however, declines more slowly with search depth: at 95% recall it is 1.74 times faster than the 56-core CPU and on par with the 112-core CPU. These CPU comparisons hold against the evaluated Faiss grid, whose only point near 95% recall uses 512 lists; Faiss configurations with more lists, which were not evaluated, could narrow or reverse them.

RQ4: How does energy compare, and what are the limits of the measurement? For a complete 100,000-query workload including initialization, the A100 system uses the least energy at every recall target, 9.3–20.1 mJ per query. The Tenstorrent system uses 30.5–46.9 mJ per query, 2.3–3.8 times the A100 value, but 53–79% less than the CPU configurations, which use 69–219 mJ. The Wormhole device itself draws only 24–34 W and accounts for 11–18% of the Tenstorrent system energy; the rest is drawn by its dual-socket host. The main limits of the measurement are that it includes initialization, so per-query values depend on workload length; that the device sensors differ, chip telemetry on Tenstorrent and board power on the A100; and that the three systems use different hosts.

RQ5: Which stages dominate, and why? Fine search accounts for 90–98% of the Tenstorrent search time at $N_{\text{list}} = 512$, the only list count for which the stages were timed; host preparation, transfers, coarse search, and output handling together take only 26–30 ms per 10,000 queries. Within fine search, batch transitions and the final aggregation overlap with the workers and are not on the critical path at these configurations; at low N_{probe} , however, the serial aggregation of 63 worker results per batch is a likely cause of the fixed per-batch floor. The time is instead spent in the per-list loop of each worker. With at most 21 floating-point operations per fetched byte, and only 0.7–1.7 useful ones, the search is memory-bound in the roofline sense and uses only 2% of the peak arithmetic rate; its time is set by irregular DRAM fetch times with shallow buffering, which create pipeline bubbles, and by the work done on every page around the multiplication, most likely the top- k update. Batch completion is further set by the slowest worker, because tasks remain coarse: whole lists, or chunks of up to 128 pages for longer lists, which occur only with 512 lists. In a schedule without chunking, which matches the timed runs at 1024 and 2048 lists, the busiest worker holds a median of 1.3–4.2 times the mean load, the most at low N_{probe} , where fewer lists than workers are available. More than any of these effects, however, the batch union multiplies the number of pages that the workers must process.

The energy hypothesis. The motivation of this thesis was that an energy-efficient machine-learning accelerator might also provide lower-energy vector search. At the system level, this is not confirmed for the evaluated setup: the A100 system uses less energy at every recall target. The measurements do, however, locate the cause. The Wormhole device is frugal: at the 95% target the chip uses 8.5 mJ per query, similar to the 10.7 mJ of the A100 board, while the system energy is dominated by the host and by the time spent in the search loop. Both can be reduced, by a lighter host and by shorter sessions, which motivates the future work below.

6.2 Limitations

The implementation is limited by the platform and by its own design in four ways.

Limited low-level primitives. The low-level kernel (LLK) API of TT-Metalium offers little room to customize the operations that the search relies on. Ordering scores together with 32-bit identifiers is possible only through a specific top- k path, which in turn requires FP32 destination accumulation (Section 3.2.2). Other primitives, such as the merge of sorted results, support only 16-bit indices, which cannot identify the 1.18 million database vectors directly. The design therefore follows what the available primitives allow rather than what the algorithm would ideally use.

Device memory. The complete index must reside in the device DRAM of the Wormhole n150, 12 GB [30]. With vectors padded to 128 FP16 dimensions and one 32×32 identifier tile per page, each database vector occupies 384 bytes, which bounds a single device to roughly 30 million vectors, far from billion-scale collections. Larger collections could in principle be swapped in and out of the device, as tiling does between DRAM and L1, but the lists would then cross the PCIe link, about 32 GB/s against 288 GB/s for the device DRAM. Since a batch union covers a large part of the database, about 374 MB at (512, 32), swapping pays off only when batches touch few lists, for example after grouping similar queries, or when the most frequently used lists are kept on the device. Such out-of-core search is usually treated as a separate problem from in-memory search: systems such as DiskANN [27] and SPANN [6] keep most of a billion-scale index on SSDs and are designed around that storage, and the NeurIPS’21 billion-scale challenge evaluated in-memory and out-of-core solutions in separate tracks [25]. This thesis addresses the in-memory setting.

Within the device, the L1 memory of each Tensix core, shared by the circular buffers, query tiles, and top-32 states, in turn limits how deeply the reader can buffer ahead of compute.

Load imbalance. Tasks are coarse: lists are split only when they exceed 128 pages, so a worker that receives a large task can still set the completion time of the whole batch, and at low N_{probe} fewer tasks than workers are available.

Partial last pages. The last page of an inverted list is usually only partially filled. Before the top- k update, its padded positions must receive a very low similarity, which the compute kernel achieves by scanning all $32 \times 32 = 1,024$ entries of the page (Section 3.2.2). Every list in a batch union pays this cost once, so it weighs most when the lists are small, as with 2048 lists.

The platforms differ in host processors and hardware generations, so the comparison is between complete systems rather than processor architectures. Energy includes initialization and uses different device sensors on the two accelerators.

6.3 Future Work

The results point to one main direction: an index designed specifically for Tenstorrent, which is expected to raise its throughput more than refinements of the current design, because the IVF scan fetches far more data than each query needs and is limited by per-page work and irregular fetch latency rather than by arithmetic. Within the current IVF design, several further steps would help: fetching less data per batch, distributing it better over the workers, and making each page cheaper to process.

1. **Design an index for the device.** The main direction, and the one that could raise throughput on the device the most, is an index designed specifically for the hardware, as CAGRA did for GPUs [20]. A graph-based index can reach high recall while visiting far fewer vectors than an IVF scan, but, as discussed in Section 2.7, a direct port of a CPU graph algorithm would not suit Tenstorrent. A graph designed for the device would take its constraints into account from the start: the limited L1 memory of each core, the tile granularity of computation, and the cost of DRAM fetches shared by many cores.
2. **Group queries before batching.** In dataset order the queries of a batch share almost no lists. Sorting them by their coarse selections before batching makes each batch fetch fewer distinct lists: in a first test at $N_{\text{list}} = 512$, with 8 partitions, fine search drops by 57% at $N_{\text{probe}} = 16$ and by 42% at $N_{\text{probe}} = 32$, for a host sorting cost of 100–180 ms per 10,000 queries. Its effect on recall remains to be measured.
3. **Use only lightweight quantization.** Quantization reduces the bytes fetched per vector, but only simple formats suit the device. Product quantization and other codebook-based schemes need a lookup table to convert each code back to a value, which does not map well onto tile operations. Scalar formats that can be converted directly, such as 8-bit or 4-bit values, are the practical options.
4. **Relieve the DRAM contention.** A high fetch volume is costly on this device because the 63 workers share six DRAM controllers with twelve GDDR6 channels [30], so their readers compete for the same channels, which is consistent with the slow fetches observed in the single-list trace. Deeper buffering does not help: a first test with an eight-page instead of a two-page reader prefetch made fine search about 7% slower, as the additional outstanding reads add to

the contention. Reducing the fetched volume is therefore the main remedy, and repeating the page-level trace at larger N_{probe} would quantify the contention directly.

5. **Distribute the work in finer slices.** The implementation already splits lists longer than 128 pages into chunks. Finer slices, down to a few pages, would balance the load further across the workers, so that no single large task sets the completion time of a batch. The single aggregation core would then remain a bottleneck only at low N_{probe} . A tree-shaped reduction is not expected to be an efficient alternative: intermediate reduction nodes would need L1 space that the fine search uses for its fetch buffers, and every core would have to act both as a worker and as a reduction node.
6. **Scale beyond one chip and one dataset.** Datasets larger than device memory, other embedding models, and multi-chip Tenstorrent systems would test whether the approach holds at the scale of heavy retrieval workloads.
7. **Integrate retrieval with inference.** The original motivation, running retrieval on the same hardware as the language model in a RAG pipeline, remains open. Co-locating both stages on one device would remove a transfer between systems and is the setting in which a Tenstorrent retrieval stage would be most valuable.

Acknowledgements

I would like to thank my advisor, Prof. Daniele De Sensi, for taking on the topic I proposed and for suggesting that I study it on emerging devices such as Tenstorrent and Cerebras. Working this way showed me what really matters for the next generation of AI inference hardware, and which trade-offs will shape the choices ahead. I am also grateful to him for arranging access to the hardware used in this work and for reading my drafts so carefully. Beyond this thesis, I thank him as a professor: the passion, competence, and care he puts into the courses I attended have been an example for me.

My sincere thanks also go to my co-advisor, Lorenzo Piarulli, for the weekly meetings and the steady support throughout the project, for the constant availability whenever I was stuck, for the many discussions about the Tenstorrent architecture, and for the detailed comments on my writing. Sharing the experience of a PhD student, Lorenzo showed me what it actually means to be a researcher and how many fields are open to explore.

A special thank you goes to my university friends, who, one semester at a time, helped me get through my exams with discussions, comparisons, and mutual support. This kind of shared effort became much rarer during my Master's studies, which is why I value my Bachelor's years even more: it was with you that I felt most challenged and motivated to grow.

A heartfelt thank you to my family, who supported me and were patient with me throughout my studies, in the easy moments and especially in the difficult ones.

Finally, thank you to my friends, for being there outside of studying and work, and for the moments that helped me recharge and keep going.

Bibliography

- [1] ALMEROL, J. L., BOELLA, E., SPERA, M., AND GREGORI, D. Accelerating gravitational N -body simulations using the RISC-V-based Tenstorrent Wormhole. arXiv:2509.19294 (2025).
- [2] AUM"ULLER, M., BERNHARDSSON, E., AND FAITHFULL, A. ANN-Benchmarks: A benchmarking tool for approximate nearest neighbor algorithms. *Information Systems*, **87** (2020), 101374. doi:10.1016/j.is.2019.02.006.
- [3] AUM"ULLER, M., BERNHARDSSON, E., FAITHFULL, A., AND ANN-BENCHMARKS CONTRIBUTORS. ANN-Benchmarks: Dataset metadata and generation code. <https://github.com/erikbern/ann-benchmarks>. Accessed: 2026-08-28.
- [4] BROWN, N. AND BARTON, R. Accelerating stencils on the Tenstorrent Grayskull RISC-V accelerator. In *International Workshop on RISC-V for HPC* (2024). ArXiv:2409.18835.
- [5] BROWN, N., DAVIES, J., AND LECLAIR, F. Exploring fast Fourier transforms on the Tenstorrent Wormhole. In *RISC-V for HPC Workshop, ISC High Performance* (2025). ArXiv:2506.15437.
- [6] CHEN, Q., ZHAO, B., WANG, H., LI, M., LIU, C., LI, Z., YANG, M., AND WANG, J. SPANN: Highly-efficient billion-scale approximate nearest neighbor search. In *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34 (2021).
- [7] CHERN, F., HECHTMAN, B., DAVIS, A., GUO, R., MAJNEMER, D., AND KUMAR, S. TPU-KNN: K nearest neighbor search at peak FLOP/s. In *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35 (2022).
- [8] GUO, R., SUN, P., LINDGREN, E., GENG, Q., SIMCHA, D., CHERN, F., AND KUMAR, S. Accelerating large-scale inference with anisotropic vector quantization. In *Proceedings of the 37th International Conference on Machine Learning*, vol. 119 of *Proceedings of Machine Learning Research*, pp. 3887–3896. PMLR (2020).
- [9] IWASAKI, M. Pruned bi-directed k-nearest neighbor graph for proximity search. In *Similarity Search and Applications*, vol. 9939 of *Lecture Notes in Computer Science*, pp. 20–33. Springer (2016). doi:10.1007/978-3-319-46759-7_2.

- [10] JÉGOU, H., DOUZE, M., AND SCHMID, C. Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, **33** (2011), 117.
- [11] JIANG, W., ET AL. Co-design hardware and algorithm for vector search. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '23)* (2023). doi:10.1145/3581784.3607045.
- [12] JOHNSON, J., DOUZE, M., AND JÉGOU, H. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, **7** (2021), 535. doi:10.1109/TBDATA.2019.2921572.
- [13] LEE, Y., CHOI, H., MIN, S., LEE, H., BEAK, S., JEONG, D., LEE, J. W., AND HAM, T. J. ANNA: Specialized architecture for approximate nearest neighbor search. In *IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pp. 169–183 (2022).
- [14] LEWIS, P., ET AL. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33 (2020).
- [15] MALKOV, Y. A. AND YASHUNIN, D. A. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, **42** (2020), 824. doi:10.1109/TPAMI.2018.2889473.
- [16] MILVUS. Hnsw. <https://milvus.io/docs/hnsw.md>. Accessed: 2026-07-30.
- [17] MILVUS. IVF-PQ. <https://milvus.io/docs/ivf-pq.md>. Accessed: 2026-07-30.
- [18] NVIDIA. NVIDIA A100 Tensor Core GPU: Product specifications. <https://www.nvidia.com/en-us/data-center/a100/>. Accessed: 2026-08-26.
- [19] NVIDIA. nvidia-smi: NVIDIA system management interface documentation. <https://docs.nvidia.com/deploy/nvidia-smi/index.html>. Accessed: 2026-09-26.
- [20] OOTOMO, H., NARUSE, A., NOLET, C., WANG, R., FEHER, T., AND WANG, Y. CAGRA: Highly parallel graph construction and approximate nearest neighbor search for GPUs. In *2024 IEEE 40th International Conference on Data Engineering*, pp. 4236–4247. IEEE (2024). doi:10.1109/ICDE60146.2024.00323.
- [21] ORACLE. Understand inverted file flat vector indexes. <https://docs.oracle.com/en/database/oracle/oracle-database/26/vecse/understand-inverted-file-flat-vector-indexes.html>. Accessed: 2026-07-30.

- [22] PENNINGTON, J., SOCHER, R., AND MANNING, C. GloVe: Global vectors for word representation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*, pp. 1532–1543. Association for Computational Linguistics (2014). doi:10.3115/v1/D14-1162.
- [23] PUGH, W. Skip lists: A probabilistic alternative to balanced trees. *Communications of the ACM*, **33** (1990), 668. doi:10.1145/78973.78977.
- [24] SHI, J., GAO, J., XIA, J., FEHÉR, T. B., AND LONG, C. GPU-native approximate nearest neighbor search with IVF-RaBitQ: Fast index build and search. *arXiv preprint arXiv:2602.23999*, (2026).
- [25] SIMHADRI, H. V., ET AL. Results of the NeurIPS’21 challenge on billion-scale approximate nearest neighbor search. In *Proceedings of the NeurIPS 2021 Competitions and Demonstrations Track*, vol. 176 of *Proceedings of Machine Learning Research* (2022).
- [26] SIVIC, J. AND ZISSERMAN, A. Video Google: A text retrieval approach to object matching in videos. In *Proceedings of the Ninth IEEE International Conference on Computer Vision (ICCV)*, pp. 1470–1477 (2003).
- [27] SUBRAMANYA, S. J., DEVVRIT, KADEKODI, R., KRISHNASWAMY, R., AND SIMHADRI, H. V. DiskANN: Fast accurate billion-point nearest neighbor search on a single node. In *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32 (2019).
- [28] TENSTORRENT. Tt-metalium documentation: Architecture and programming model. <https://docs.tenstorrent.com/tt-metal/latest/tt-metalium/>. Accessed: 2026-07-30.
- [29] TENSTORRENT. TT-SMI: Tenstorrent console based hardware information program. <https://github.com/tenstorrent/tt-smi>. Accessed: 2026-09-26.
- [30] TENSTORRENT. Wormhole pcie cards: Hardware specifications. <https://docs.tenstorrent.com/aibs/wormhole/index.html>. Accessed: 2026-08-26.
- [31] TURISINI, M., CESTARI, M., AND AMATI, G. LEONARDO: A pan-european pre-exascale supercomputer for HPC and AI applications. *Journal of Large-Scale Research Facilities*, **9** (2024), A186. doi:10.17815/jlsrf-8-186.